Team 41

VitalTrack: Smart Remote Patient Monitoring with AI-Based Risk Prediction and a Health Chatbot

VitalTrack is a smart remote patient monitoring platform that collects patient vital signs from connected devices, stores and analyzes them, detects abnormal conditions, generates alerts and AI-based risk predictions, and provides secure access through dashboards, patient portals, and a health chatbot.

CODE Repo with README : https://github.com/usr1236/project3_41.git , https://github.com/usr1236/project3_41

Task 1: Requirements and Subsystems

1. Functional Requirements

FR-01 – Vital Sign Data Collection
The system shall collect vital sign data from wearable devices, sensors, or simulators through secure interfaces.

FR-02 – Data Persistence
The system shall persist incoming health data and related events for monitoring, analysis, auditing, and future reference.

FR-03 – Doctor Monitoring Dashboard
Doctors shall be able to monitor multiple patients' vital signs through a near real time dashboard, with typical end-to-end update latency of approximately 2 seconds under normal operating conditions.

FR-04 – Patient Portal Access
Patients shall be able to securely access their own personal health information, historical vital sign data, alerts, and generated reports through a personalized portal.

FR-05 – Abnormal Condition Detection and Alerting
The system shall detect abnormal vital sign patterns using configurable clinical rules and thresholds, generate alert records with severity levels, and support alert deduplication, suppression, and acknowledgment workflows where applicable.

FR-06 – Critical Notification Delivery and Escalation

The system shall deliver critical or high-severity alerts to doctors, patients, caregivers, or family members through configured notification channels such as in-app notifications, email, or SMS. For critical alerts, the system shall support escalation when an alert remains unacknowledged beyond a configured escalation window, notifying the next configured recipient in sequence (e.g., primary doctor, on-call doctor, caregiver/family member, administrator) until the alert is acknowledged or the escalation chain is exhausted.

FR-07 Risk Prediction and Early Warning

The system shall analyze patient vital signs to produce advisory risk predictions that augment absolute threshold-based alerting. Risk prediction shall combine (a) clinical early-warning scoring using population-level thresholds inspired by the NEWS2 standard, and (b) patient-specific statistical anomaly detection using rolling baselines with z-score deviation. Prediction outputs shall be advisory in nature and shall not replace clinician judgment. The subsystem shall be extensible to support additional prediction strategies, including future ML-based models, without requiring changes to other subsystems.In the prototype, high-risk prediction outputs may be configured to generate supplementary alert records to enhance early warning capabilities.

FR-08 – Health Chatbot

The system shall provide a direct in-application chatbot endpoint for non-diagnostic guidance and triage support. The chatbot classifies message risk levels (LOW/MEDIUM/CRITICAL) and returns advisory responses to the user. Chatbot outputs are advisory in nature, do not create alerts or trigger notification or escalation workflows, and do not replace clinician judgment. The chatbot uses a provider strategy order (LLMs→ deterministic local rules fallback), enabling resilience when external providers are unavailable.

FR-09 – Authentication and Role-Based Authorization

The system shall require secure authentication and enforce role-based access control for patients, doctors, caregivers, and administrators.

FR-10 – Administration and Audit Logging

Administrators shall be able to manage users, devices, thresholds, and system settings, and the system shall maintain audit logs of key operational and user activities, including access to patient data, configuration changes, authentication events, and alert-related actions.

2. Non-Functional Requirements

(Prototype targets and production targets are distinguished inline.)

NFR-01 Performance

The system shall process incoming health data and update the monitoring dashboard within approximately 2 seconds of data receipt under normal conditions in production, with a prototype target of approximately 3 to 5 seconds under limited load. For typical synchronous API requests, the system shall target a 95th-percentile response time of 500 ms or lower in production, with a prototype target of 1 second or lower. The system shall support near real time delivery of monitoring updates via push-based mechanisms. **Response measures:**

- End-to-end alert latency (production target): median ≤1 second, p95 ≤2 seconds from ingestion to dashboard delivery.
- End-to-end alert latency (prototype target): median ≤3 seconds, p95 ≤5 seconds under limited load; actual measured value reported in Task 4.
- Synchronous API response time: p95 ≤1 second (prototype); p95 ≤500 ms (production).
- Ingestion throughput: prototype target is to measure maximum sustainable ingestion throughput under controlled test load and report the observed value in Task 4. Production design target is 100 vital readings per second per ingestion component.

NFR-02 Availability and Reliability

The production system shall target 99.9% uptime, while the prototype shall target stable availability in a controlled demonstration environment rather than a formal uptime commitment. The system shall minimize data loss and ensure reliable processing of health data events, including tolerance to duplicate delivery and safe reprocessing. The system shall support retry handling, failed event handling, and failure isolation. **Response measures:**

- Uptime: 99.9% in production; stable operation for the duration of a controlled demonstration in the prototype.
- Event loss rate: ≤0.1% of published alert events lost end-to-end, measured as events that never reach downstream consumers.
- **Fault tolerance (continued processing during partial failure):** the ingestion path continues to accept and persist new vital readings and alerts during a message-broker outage; events accumulate in the outbox and are published automatically once the broker is reachable.
- Retry policy: failed event deliveries retried at least 3 times before being marked as exhausted; the majority of transient failures should resolve within those retries.

- Recovery: after a message-delivery outage, all undelivered events shall be processed within 60 seconds of system recovery.

NFR-03 Security

The system shall protect sensitive data using secure communication protocols such as TLS 1.2 or higher, encryption for data at rest such as AES-256, and secure credential handling such as bcrypt or equivalent. In the prototype, some controls may be demonstrated in simplified form, while production deployments shall enforce them comprehensively across all interfaces. The system shall enforce authentication and authorization across all interfaces and protect exposed endpoints using access control mechanisms. **Response measures:**

- Authentication response time: invalid credentials rejected within 200 ms (200-500ms for prototype).
- Credential storage: passwords stored using strong one-way hashing; session tokens protected by cryptographically strong signing keys.
- Audit and detection: authentication events, alert lifecycle transitions, and admin operations are captured in the audit trail (see NFR-04 for full audit coverage).

NFR-04 Privacy

The system shall enforce least-privilege access to patient data and maintain audit trails of access to sensitive information. The architecture shall support HIPAA-aligned technical safeguards in production deployments and enable compliance with applicable healthcare privacy and data protection regulations. In the prototype, these safeguards are demonstrated architecturally and partially implemented rather than claimed as full compliance. Regulatory compliance additionally requires administrative and physical controls beyond software design alone. **Response measures:**

- Audit coverage: sensitive data access events (authentication, alert lifecycle transitions, admin operations, caregiver approvals, cross-patient reads) recorded in the audit trail with actor, action, target, and timestamp.
- Access scoping: patient data endpoints return an authorization-denied response for any user outside the patient's authorized access set (owner, assigned doctor, approved caregiver, admin); verified through automated tests covering representative role and access combinations.
- PII in application logs: no patient names, vital values, or other PHI appear in plaintext application logs; audit records use internal identifiers and structured metadata only.
- Data retention: retention periods shall be configurable according to applicable healthcare and organizational policies; the prototype uses indefinite retention for demonstration purposes.

NFR-05 Scalability

The system shall support horizontal scaling to accommodate growth in patients, devices, and concurrent users, with a production target of up to 10,000 concurrent patients and a prototype target focused on smaller controlled workloads suitable for demonstration and validation. It shall also support independent scaling of background processing workloads such as alerting, prediction, and notification delivery. **Response measures:**

- Concurrency targets: prototype supports at least 50 concurrent users and 100 active patients on a single-instance deployment; production targets 10,000 concurrent patients with horizontal scaling.
- Background worker independence: alerting, notification delivery, and future prediction workloads shall be able to scale independently of the ingestion request path.

NFR-06 Modifiability and Maintainability

The system shall follow a modular architecture that allows new device integrations, analytical capabilities, and notification channels to be introduced with minimal changes to existing subsystems. The system shall support controlled updates and maintainability through clear subsystem boundaries, interface contracts, and logging. **Response measures:**

- Change locality: a typical new feature (new clinical rule, new notification channel, new device adapter) requires changes to at most two subsystems and no breaking changes to stored data schemas.
- New event consumer: adding a new subsystem as an additional event subscriber requires zero changes to existing services. Demonstrated by the introduction of the Prediction service, which consumes existing VITAL_RECEIVED events without changes to Ingestion or Notification.

NFR-07 Testability

Each subsystem shall expose well-defined interfaces to support automated testing. The production target is at least 80% line coverage for critical modules. The prototype target is meaningful coverage of architecturally significant and high-risk paths through unit and integration testing, rather than a fixed coverage percentage; actual coverage achieved in the prototype is reported in Task 4. **Response measures:**

- Production coverage target: ≥80% line coverage on critical modules (modules list specified in Task 4).
- Prototype coverage target: at least one integration test per implemented end-to-end flow and at least one unit test per critical function (risk scoring, alert evaluation, access-scope enforcement).

NFR-08 Usability

The system interface shall be accessible, intuitive, and user-friendly for all user types, including elderly or non-technical patients. The system shall target a System Usability Scale (SUS) score of at least 70

for production readiness, while the prototype shall gather initial usability feedback to validate design direction. **Response measures:**

- Production target: SUS score ≥70 with at least 10 representative users including elderly patients and clinicians.
- Prototype target: informal usability check with at least 3 users across different roles to validate design direction.

NFR-09 Interoperability

The architecture shall preserve extension points for future integration with external healthcare systems through standardized data exchange mechanisms such as HL7 FHIR. This capability is a future extension and is outside the scope of the current prototype. Within the prototype, API contracts are explicitly versioned and documented to preserve the ability to evolve them without breaking consumers. **Response measures:**

- API versioning: all externally exposed APIs are explicitly versioned and published with machine-readable specifications.
- Event versioning: domain events carry explicit schema-version and event-type metadata, allowing consumers to evolve independently.
- FHIR integration: preserved as an architectural extension point for future integration.

: Quality Attribute Scenarios

The scenarios below make the architecturally significant NFRs testable by specifying source, stimulus, artifact, environment, response, and response measure.

QAS-01 Performance (Alert Delivery Latency)

- **Source:** External device or simulator
- **Stimulus:** A vital-sign reading crossing a critical threshold is submitted to the ingestion interface
- **Artifact:** Ingestion subsystem, asynchronous event-delivery channel, notification subsystem, monitoring dashboard
- **Environment:** Normal operating conditions, single-instance services, prototype deployment
- **Response:** An alert is created, persisted, delivered to all connected doctor dashboards via push-based updates, and recorded in the audit trail
- **Response Measure:** Median end-to-end latency ≤3 seconds; p95 ≤5 seconds (prototype); no alert events lost across the tested ingestion workload
- **Related NFRs:** NFR-01, NFR-02

QAS-02 Availability (Event-Delivery Recovery)

- **Source:** Internal infrastructure failure
- **Stimulus:** The event-delivery channel becomes unreachable for 60 seconds and then recovers
- **Artifact:** Ingestion subsystem, event-delivery channel, notification subsystem
- **Environment:** Continuous ingestion load (approximately 5 readings per second) throughout the outage
- **Response:** Vital ingestions succeed and are durably persisted during the outage. After recovery, undelivered events are processed and delivered to downstream consumers
- **Response Measure:** Zero alert events lost; undelivered events processed within 60 seconds of recovery
- **Related NFRs:** NFR-02

QAS-03 Security (Unauthorized Access Attempt)

- **Source:** External unauthenticated or under-privileged user
- **Stimulus:** Request to a protected endpoint with a missing, expired, under-privileged, or tampered session token
- **Artifact:** External entry point, authentication and authorization components, endpoint handler
- **Environment:** Normal operating conditions
- **Response:** Request is rejected with an appropriate authorization-denied response; no sensitive data is returned
- **Response Measure:** All unauthorized access attempts rejected in automated tests; rejection occurs within 200 ms; security-relevant rejections are captured in the audit trail where audit logging is enabled
- **Related NFRs:** NFR-03, NFR-04

QAS-04 Privacy (Patient Data Scope Violation)

- **Source:** Authenticated user attempting unauthorized access
- **Stimulus:** Request for another patient's data by a user not authorized for that patient
- **Artifact:** Patient data endpoint, access-control component, data store
- **Environment:** Normal operating conditions
- **Response:** Request is rejected with an authorization-denied response; the attempt is recorded in the audit trail
- **Response Measure:** All cross-patient access attempts blocked in automated tests; no PHI leaked in responses or application logs; audit entry contains actor, attempted target, and timestamp
- **Related NFRs:** NFR-03, NFR-04

QAS-05 Modifiability (Adding a New Notification Channel)

- **Source:** Developer

- **Stimulus:** Requirement to add a new notification channel (e.g., SMS) alongside existing channels
- **Artifact:** Notification subsystem
- **Environment**: Build time
- **Response:** The new channel is introduced without modifying ingestion, external entry points, or event contracts
- **Response Measure:** Changes confined to the notification subsystem; existing functionality unaffected; the new channel works end-to-end after integration
- **Related NFRs:** NFR-06

QAS-06 Testability (End-to-End Flow Validation)

- **Source:** Developer
- **Stimulus:** Code change followed by execution of the integration test suite
- **Artifact:** Full system (all subsystems involved in end-to-end flows)
- **Environment:** Containerized test environment
- **Response:** End-to-end flows are executed; failures in any subsystem cause test failure
- **Response Measure:** At least one integration test per end-to-end flow; at least one unit test per critical function; the suite detects failures in alert creation, event delivery, notification, or audit logging
- **Related NFRs:** NFR-07

QAS-07 Modifiability (Introducing a New Prediction Strategy)

- Source: Developer or data scientist
- Stimulus: Requirement to add a new risk-prediction strategy (e.g., an ML-based classifier)
- Artifact: Prediction Service, strategy interface
- Environment: Build time
- Response: New strategy implemented as an additional class behind the existing interface. Existing EWS and Personalized Baseline strategies remain unchanged. No changes to Ingestion, Notification, or event schemas.
- Response Measure: Changes confined to the Prediction Service. Existing end-to-end tests pass. New strategy exercisable end-to-end.
- Related NFRs: NFR-06, ASR-05.

QAS-08 Critical Alert Escalation

- **Source:** Internal monitoring workflow
- **Stimulus:** A critical alert remains unacknowledged for the configured escalation interval.
- **Artifact:** Escalation Mediator service, alert state store, notification channel, audit log
- **Environment:** Normal runtime with active broker and persistence
- **Response:** The mediator escalates to the next configured recipient and records escalation actions in the audit trail; escalation continues until acknowledged or recipients are exhausted
- **Response Measure:** Escalation occurs within 5 seconds of the configured timeout expiring; each escalation step is recorded in the audit trail with timestamp, recipient, and outcome; the workflow completes or terminates within the configured maximum escalation duration
- **Related NFRs:** NFR-02, NFR-06

QAS-09 Chatbot Advisory Response

- **Source:** Patient or Caregiver
- **Stimulus:** User submits a symptom-related message to the chatbot
- **Artifact:** Chatbot endpoint, Notification Service, audit log
- **Environment:** Normal runtime
- **Response:**
  Chatbot classifies the message risk level and returns an advisory response to the user. The interaction is recorded in the audit trail.
- **Response Measure:**
  Chatbot returns a response within the prototype latency target and logs the interaction with classification, timestamp, and advisory flag. No alert, notification, or escalation workflow is triggered.
- **Related NFRs:** NFR-01, NFR-04, NFR-07

---

3. Architecturally Significant Requirements (ASRs)

ASR-01 Near Real Time Data Processing (FR-01, FR-03, NFR-01) The system must handle continuous incoming health data and make it available to monitoring dashboards within approximately 2 seconds in production, with a looser prototype target acceptable during initial validation, ensuring low-latency data availability for near real time monitoring.

ASR-02 Security and Privacy (FR-09, NFR-03, NFR-04) The system must enforce strong authentication, authorization, encryption, and auditability to protect sensitive healthcare data and support privacy compliance.

ASR-03 Availability and Reliability (NFR-02) The system must maintain high availability with a production target of 99.9% uptime and ensure reliable processing of health data and alerts, including tolerance to failures, retries, and safe handling of repeated or delayed events through idempotent processing and retry mechanisms

ASR-04 Modifiability (NFR-06) The architecture must support the addition of new devices, updated prediction capabilities, and new notification or integration features with minimal disruption to existing subsystems.

ASR-05 AI/ML Prediction Capability (FR-07, NFR-06) The system must support predictive analysis of patient data with the ability to evolve models, update prediction logic, and maintain traceability of prediction outputs and model versions. This is architecturally significant because prediction logic is expected to change independently of the rest of the system, requiring strong modifiability boundaries around the AI Prediction subsystem.

ASR-06 Interoperability and External Integration (NFR-09) The architecture must preserve extension points for future integration with external healthcare systems and standards-based interfaces while keeping those integrations outside the scope of the current prototype.

4. Subsystem Overview

SS-01 Data Ingestion Subsystem Handles data acquisition from wearable devices, sensors, or simulators. Validates, normalizes (format standardization), enriches, and forwards incoming health data for downstream processing.

SS-02 Patient Data Management Subsystem Manages persistent business data including patient profiles, vital sign histories, alerts, reports, and prediction records. Supports storage, retrieval, querying, and reporting over health-related data.

SS-03 Alert and Notification Subsystem

Monitors incoming vital sign data against configurable clinical rules and threshold-based criteria. When abnormal readings are detected, it generates alert records, assigns severity levels, manages alert state, supports acknowledgment workflows, and delivers notifications to relevant recipients through configured channels. AI-based risk predictions are handled by the Risk Prediction Subsystem and are presented as advisory signals; high-risk prediction outcomes may trigger supplementary risk alerts based on configured policies, without replacing primary threshold-based alert workflows.

SS-04 Risk Prediction Subsystem

Consumes vital-sign events from the ingestion stream and produces advisory risk predictions. Implements two complementary strategies:

A clinical early-warning score inspired by NEWS2 that assigns points per vital range and aggregates to an overall severity score using population-level clinical thresholds.
A patient-specific statistical baseline that tracks rolling mean and standard deviation of each vital per patient and flags deviations exceeding a configurable z-score threshold.
Prediction events include score values, contributing factors, strategy version, and timestamp, and are published back to the event stream for downstream consumers (notification, audit, future subsystems). The subsystem is designed as an independent service with no synchronous dependencies on other services.

SS-05 Chatbot Capability (Integrated in Notification Service)

Provides a conversational triage interface via backend API. Performs keyword- and model-based risk classification, returns advisory responses, and records audit entries for all interactions. Implemented as an integrated API capability within the Notification Service in the prototype (not a separate microservice).

Chatbot outputs are advisory-only and do not trigger alert creation, notification delivery, or escalation workflows. The chatbot uses a provider strategy order (OpenAI / Gemini → deterministic local rules fallback) and normalizes responses into a consistent internal format.

SS-06 Web Interface / Dashboard Subsystem Provides role-specific user interfaces for patients, doctors, caregivers, and administrators, supporting monitoring, visualization, reporting, alert review, and interaction.

SS-07 Authentication and Authorization Subsystem Manages identity, authentication, session security, and role-based access control for all system users.

SS-08 Administration and Monitoring Subsystem Provides tools for managing users, devices, thresholds, and system configurations, and supports audit logging, operational monitoring, and administrative oversight.

SS-09 API Access Subsystem Serves as the logical application-layer entry point for client applications and exposes backend APIs. Coordinates request handling and applies application-level cross-cutting concerns such as authentication-context propagation, input validation, and request routing . (distinct from the external API Gateway)

SS-10 Interoperability Subsystem (Future Extension) Represents the extension point for future integration with external hospital systems and standardized healthcare data exchange interfaces. Outside the scope of the current prototype.

SS-11 Escalation Mediator Subsystem

Coordinates multi-step escalation workflows for critical alerts that remain unacknowledged. Subscribes to ALERT_CREATED events, maintains workflow state per active escalation, advances through the escalation chain (primary doctor → on-call doctor → caregiver/family → admin) with timed waits, and records each step in the audit log. Workflow state is persisted to support recovery across restarts.

Task 2: Architecture Framework

Following the IEEE 42010 standard, this section identifies the system's stakeholders, their concerns, and the architectural viewpoints and views used to address those concerns.

2.1 Stakeholders and Concerns

Stakeholder	Concerns
Patients	Easy access to personal data, timely alerts, chatbot support, privacy and security
Doctors / Healthcare Providers	Near real time monitoring, data accuracy, alert reliability, prediction traceability
Caregivers / Family Members	Timely high-severity notifications, simplicity
System Administrators	Uptime, maintenance, user/device/policy management, observability

Development Team	Modularity, testability, clear boundaries, API clarity, extensibility
Regulatory / Compliance Stakeholders	Encryption, audit trails, access control, privacy-by-design
Hospital Administrators	Integration readiness, operational cost, training, ROI
Operations and Support Team	Monitoring, incident response, updates, backups, SLO adherence
Clinical Team / On-Call Doctors	Reliable alert delivery, escalation handling, low false positives

2.2 Architectural Viewpoints and Views

VitalTrack adopts a comprehensive viewpoint set inspired by Rozanski and Woods, augmented with a Use Case (Scenarios) viewpoint from Kruchten's 4+1 model.

Viewpoint	Description	Stakeholders Addressed	Views / Artifacts
Functional Viewpoint	External behavior and responsibilities, including collection, monitoring, alerting, prediction, chatbot, and administration. Chatbot functionality is delivered as an API capability within the Notification Service rather than a	Patients, Doctors, Hospital Administrators, Development Team	Functional decomposition, context diagrams, data-flow diagrams, API specifications

	separate subsystem in the prototype.		
Logical Viewpoint	Internal decomposition into modules, subsystems, and relationships	Development Team, System Administrators	Subsystem diagrams, component diagrams, UML class diagrams
Information Viewpoint	Data storage, flow, and lifecycle for patients, vitals, alerts, reports, logs, and predictions	Patients, Regulatory Stakeholders, Development Team	ER diagrams, data lifecycle diagrams, data ownership and retention diagrams
Process / Runtime Viewpoint	Runtime interactions, including asynchronous event processing, retries, idempotency, WebSocket updates, and chatbot interaction flows (user message → chatbot triage → advisory response returned to user with audit logging).	Doctors, Development Team, Operations Team	Sequence diagrams, activity diagrams, runtime interaction diagrams
Deployment Viewpoint	Mapping to infrastructure such as gateway, four backend service instances (Ingestion, Notification, Prediction, Escalation Mediator), PostgreSQL with TimescaleDB, message broker, and notification providers. The gateway handles request routing and WebSocket	System Administrators, Hospital Administrators, Operations Team, Development Team	Deployment diagrams, topology diagrams, dependency diagrams

	upgrade in the prototype; TLS termination, rate limiting, and other edge concerns are deferred to production deployment.		
Security Viewpoint	JWT-based authentication, role-based authorization, credential hashing, audit logging, and secure transport	Patients, Regulatory Stakeholders, System Administrators, Operations Team	Security diagrams, access matrices, threat models
Operational Viewpoint	Monitoring, logging, incident response, backups, and queue operations	Operations Team, System Administrators, Hospital Administrators	Dashboards, runbooks, SOPs, SLO definitions
Use Case (Scenarios) Viewpoint	End-to-end actor interactions as the “+1” integrative view	Patients, Doctors, Caregivers, Hospital Administrators, Development Team	Use case diagrams, narratives, user journeys

2.3 Stakeholder – Concern – Viewpoint Mapping

Stakeholder	Key Concerns	Addressed By Viewpoint
Patients	Privacy, usability, chatbot access, alerts	Functional, Information, Use Case, Security

Doctors / Healthcare Providers	Near real time monitoring, data accuracy, prediction insights	Functional, Process / Runtime, Use Case
Caregivers / Family Members	Timely emergency notification, simplicity	Functional, Process / Runtime, Use Case
System Administrators	System health, management, uptime, policy enforcement	Deployment, Security, Operational
Development Team	Modularity, testability, maintainability, boundaries	Logical, Functional, Information, Use Case
Regulatory / Compliance Stakeholders	Privacy safeguards, encryption, auditability	Security, Information
Hospital Administrators	Cost, integration, operational fit	Deployment, Operational, Use Case
Operations and Support Team	Monitoring, incident response, backups, maintenance	Operational, Deployment, Process / Runtime
Clinical Team / On-Call Doctors	Alert reliability, escalation handling	Functional, Process / Runtime, Use Case

Architectural Decision Records (ADRs)

ADR-001: Use Event-Driven Architecture with a Message Broker and Pipe-and-Filter Ingestion

Status: Accepted

Context: VitalTrack continuously processes incoming vital-sign data from devices. Multiple downstream capabilities, including dashboard updates, alerting, AI prediction, and notifications, consume the same stream at different rates. A fully synchronous approach would increase coupling and reduce resilience. Ingestion itself also requires staged transformations such as validation, normalization, enrichment, and publishing, which fit a pipe-and-filter flow.

Decision: Adopt an event-driven architecture using RabbitMQ as the message broker for the prototype. Data Ingestion publishes VITAL_RECEIVED events for every accepted reading and ALERT_CREATED events when alert rules fire. The Prediction service consumes VITAL_RECEIVED events and publishes RISK_PREDICTED events. The Notification service consumes both ALERT_CREATED and RISK_PREDICTED events. Downstream services consume them asynchronously using publish-subscribe with acknowledgment, retry, and Dea-letter handling. Ingestion processing follows a pipe-and-filter pattern across transformation stages (validation, normalization, enrichment, publishing). Apache Kafka is noted as a production alternative if throughput or replay requirements grow beyond RabbitMQ's comfortable operating range, but is out of scope for the prototype.

Topology: VitalTrack uses a hybrid EDA topology, combining Broker and Mediator patterns based on flow characteristics.

The **Broker topology** (choreography-style) is the default and applies to simple reactive flows where one event triggers one or more independent consumers with no cross-consumer coordination. Examples in VitalTrack include vital-sign ingestion → alert evaluation, alert creation → notification + WebSocket broadcast + audit, and the parallel reaction of the Prediction service to incoming vitals (and, in planned future extensions). These flows favor choreography because the consumers are independent, the logic is stateless per event, and adding new consumers must not require modifying existing ones.

The **Mediator topology** (orchestration-style) is introduced selectively for flows that are multi-step, stateful, and conditionally branching. The planned example is a critical-alert escalation workflow, in which an unacknowledged critical alert must be retried across a sequence of recipients (primary doctor → on-call doctor → emergency contact → admin) with timing, conditional progression, and state preserved across steps. Without a mediator, this logic would be duplicated across consumers, re-coupling them. A dedicated escalation mediator service owns the workflow state and coordinates the steps.

The prototype implements Broker flows and a limited Mediator flow for critical-alert escalation.

Consequences: This improves decoupling, independent scaling, and burst tolerance. It simplifies adding new consumers and pipeline stages. It also adds operational complexity around ordering, duplicates, idempotency, retries, failed event handling, schema governance, and observability. The broker becomes a critical component that must be durable, monitored, and appropriately sized.

ADR-002: Use PostgreSQL with TimescaleDB Extension for Data Storage

Status: Accepted

Context: The system must handle both relational application data and high-frequency time-series vital data. Splitting storage technologies early increases operational overhead and integration complexity.

Decision: Use PostgreSQL as the primary data platform with the TimescaleDB extension for time-series workloads. Use relational tables for core business entities and hypertables for timestamped vital streams.

Consequences: This provides a unified data platform with strong consistency and mature tooling. It reduces operational fragmentation. It requires explicit TimescaleDB installation and configuration, migration DDL for hypertables, and tuning of chunk intervals, indexes, and retention policies. The decision remains reversible if scale later favors separate TSDB adoption.

ADR-003: Use WebSockets for Near Real-Time Data Delivery

Status: Accepted

Context: Doctors and caregivers require low-latency live monitoring. Polling creates a latency-versus-load trade-off, especially at higher concurrency.

Decision: Use WebSockets for push-based real-time updates to dashboards, implemented through FastAPI's native WebSocket support. Keep REST APIs for request-response flows such as authentication, historical queries, and administration.

Consequences: This improves responsiveness and reduces unnecessary polling traffic. It requires persistent connection lifecycle handling, including authentication, reconnect behavior, and fallback handling. Multi-instance scaling requires shared pub-sub propagation, such as Redis (in future production) Pub/Sub or broker relay, rather than relying solely on sticky sessions.

ADR-004: Introduce an API Gateway as a Single External Entry Point

Status: Accepted

Context: VitalTrack exposes sensitive healthcare APIs across multiple roles. Security must be consistently enforced at the edge while preserving service-level fine-grained authorization. With the microservices deployment adopted in ADR-006, a single external entry point becomes important for consistent routing and policy enforcement.

Decision: Introduce an API Gateway as the single external entry point for all client traffic. For the prototype, the gateway is implemented as a lightweight reverse proxy (nginx) handling routing to the appropriate backend

service, including Ingestion, Notification, Prediction, and Escalation Mediator and supports WebSocket upgrade. TLS termination, coarse-grained token validation at the edge, and rate limiting are production concerns and are not implemented in the prototype gateway. Each backend service independently enforces authentication and fine-grained RBAC using the JWT presented on every request.

Consequences: This centralizes edge security and traffic governance, reducing duplicated policy logic across services. It adds a critical component that must be highly available, monitored, and carefully configured to avoid broad blast radius from misconfiguration.

ADR-005: Use FastAPI for Backend Services

Status: Accepted

Context: The project requires a backend capable of exposing REST endpoints, consuming broker messages, handling WebSocket connections, and integrating with PostgreSQL. With the microservices deployment adopted in ADR-006, the backend stack must support rapid service-level iteration and a lightweight per-service footprint. Spring Boot was considered but rejected for the prototype because its footprint and ceremony are disproportionate to a four-service prototype, and the team can deliver faster on a uniform Python stack.

Decision: Implement each backend service using FastAPI (Python) with SQLAlchemy for persistence, pika for RabbitMQ integration, and FastAPI's native WebSocket support. A uniform backend stack is applied across services to simplify the prototype.

Consequences: FastAPI provides mature async support, type-hint-driven validation, and a lightweight per-service footprint suitable for microservices deployment. A uniform stack accelerates onboarding and reduces the number of build pipelines. The trade-off is that a single-language stack forgoes the mixed-technology flexibility that microservices otherwise permit; this is an accepted constraint for the prototype.

ADR-006: Microservices Deployment (Coarse-Grained, Service-Oriented Prototype Split) with Event-Driven Communication

Status: Accepted

Context:
VitalTrack’s subsystems have different workload, scaling, and evolution characteristics. Ingestion handles continuous high-volume device data, while notification and alerting operate asynchronously with retry and failure-handling requirements. AI prediction and chatbot capabilities are expected to evolve independently. A single deployable would force all components to scale and fail together. The event-driven architecture (ADR-001) provides the foundation for decoupling these responsibilities.

Decision:
Deploy VitalTrack as a coarse-grained, service-oriented microservices architecture, where multiple related responsibilities are grouped within each service to reduce prototype complexity while preserving a path toward finer-grained decomposition in production. The prototype adopts a coarse-grained, service-oriented split with four backend services.

For the prototype:

Ingestion Service — handles vital data ingestion, validation, persistence, threshold-based alert evaluation (risk-based alerts are produced by the Prediction service from RISK_PREDICTED events) , and publishes VITAL_RECEIVED and ALERT_CREATED events. It also hosts the /auth/token endpoint (a prototype compromise; a dedicated auth service is planned for production). Notification Service - consumes ALERT_CREATED and RISK_PREDICTED events, manages alert lifecycle and acknowledgment, dispatches notifications, performs retry handling, broadcasts WebSocket updates, and records audit events. Prediction Service — subscribes to VITAL_RECEIVED events from the message broker, runs early-warning and personalized-baseline strategies, and publishes RISK_PREDICTED events back to the broker. Maintains per-patient baseline state in the shared database. Escalation Mediator Service - consumes ALERT_CREATED, tracks critical unacknowledged alerts, and coordinates multi-step escalation (primary doctor -> on-call doctor -> caregiver/family -> admin) with timed retries and audit logging.

Services communicate asynchronously through RabbitMQ using a publish–subscribe model. Synchronous inter-service REST calls are minimized to maintain loose coupling. JWT-based authentication is issued by the ingestion service and validated independently by each service.

Additional capabilities such as a dedicated Auth service are designed as independent services to be added post-prototype , consuming the same event stream without changes to existing services. Service discovery is handled via Docker Compose DNS in the prototype, with a dedicated registry reserved for future scaling.

The prototype uses a shared PostgreSQL with TimescaleDB instance with logical table separation per service. This reduces operational overhead but introduces schema-level coupling. In production, services will move to independent databases or schemas, with downstream services maintaining their own read views derived from the event stream.

The four services share a common Python codebase with separate entry points, reducing duplication but introducing a shared release dependency. Production deployment will separate services into independent codebases with a shared contract library for event schemas.

The direct chatbot triage endpoint (`/v1/chatbot/message`) is also implemented within the Notification Service for the prototype.

Consequences:
This design enables independent scaling of ingestion, notification, prediction, and escalation workloads, failure isolation, and incremental addition of new services through event subscription. Trade-offs include increased operational complexity, reliance on eventual consistency, distributed debugging challenges, and reduced independence due to the shared database and shared codebase in the prototype.

ADR-007: Use React Single Page Application for Prototype UI

Status: Accepted

Context:
The prototype requires a user interface to exercise the implemented end-to-end flows (critical alert delivery and doctor acknowledgment) across multiple roles (patient, doctor, caregiver, admin, simulator). A minimal static HTML approach was initially considered but proved insufficient for supporting role-based dashboards, token handling, real-time updates, and data visualization.

Decision:
Implement the prototype UI as a React single-page application using Vite for build tooling and react-router for role-based client-side routing. The application is served as a static bundle by a dedicated frontend container behind the API gateway. All communication with backend services occurs through the gateway, and the frontend has no direct knowledge of backend service topology. Client-side role guards are used for routing and user experience, while all authentication and authorization are enforced server-side.

Consequences:
The UI supports role-based dashboards, real-time WebSocket updates, and basic visualization of vital data. It introduces a separate frontend build pipeline and container, as well as a dependency on the gateway for routing. Client-side role parsing from JWT tokens is used only for user experience purposes; backend services remain the authoritative source for access control and data protection.

ADR-008: Transactional Outbox for Reliable Event Publishing

Status: Accepted

Context:
The ingestion service must persist domain state (e.g., alerts) and publish corresponding events to the message broker. A direct publish-after-commit approach is not atomic: if the broker is unavailable at publish time, the database commit succeeds but the event is lost. Conversely, publish-before-commit risks consumers receiving events for data that was never committed. This creates a dual-write consistency problem in an event-driven architecture.

Decision:
Adopt the transactional outbox pattern. Within a single database transaction, both the domain change (e.g., alert record) and a corresponding entry in an `outbox_events` table are written. A background worker reads pending outbox records and publishes them to the message broker. Upon successful publish, the event is marked as completed; failed events are retried with a bounded retry policy.

Consequences:
This ensures that no committed domain event is lost due to temporary broker unavailability and guarantees at-least-once delivery. It introduces slight latency due to asynchronous publishing and requires monitoring of the outbox table. In multi-instance deployments, coordination (e.g., row-level locking or `SKIP LOCKED`) is needed to avoid duplicate publishing.

ADR-009: Admin-Mediated Caregiver-to-Patient Assignment

Status: Accepted

Context:
Caregivers need access to patient data, but allowing direct self-assignment introduces a privacy risk. Without control, a malicious or incorrect user could claim access to arbitrary patients, violating confidentiality requirements.

Decision:
Introduce an admin-mediated approval workflow. Caregivers submit a request to link to a patient, which remains in a pending state. An administrator reviews and approves or rejects the request. Only approved relationships grant access to patient data, enforced through role-based access control and access-scoping logic.

Consequences:
This approach ensures controlled access to sensitive patient data and prevents unauthorized self-claims. It improves privacy and compliance but introduces operational overhead and slight onboarding delays. Future enhancements may include patient-driven approvals or stronger identity verification.

Note on prototype self-registration: For demonstration purposes, the prototype's `/auth/register` endpoint accepts `PATIENT`, `CAREGIVER`, and `DOCTOR` role self-registration. In production, DOCTOR accounts would be created only by administrators through the credentialing workflow (verified licensing, institutional affiliation, background checks). The self-registration allowlist for DOCTOR is a deliberate prototype simplification that allows markers and evaluators to exercise doctor-role flows without administrator setup. The admin-mediated caregiver workflow described above is unaffected.

ADR-010: Versioned Event Envelope for Broker Messages

Status: Accepted

Context:
Multiple services consume events from the message broker and may evolve independently. Without explicit versioning, schema changes in event payloads can break consumers and require coordinated deployments, reducing system flexibility.

Decision:
Define a versioned event envelope structure for all published events, including fields such as:

- `schema_version`
- `event_type`
- `occurred_at`
- `data` (payload)

Consumers use `schema_version` and `event_type` to determine how to process events and can safely ignore unsupported versions.

Consequences:
This enables independent evolution of producers and consumers, supports backward compatibility, and allows rolling upgrades. It introduces minimal payload overhead and requires discipline in maintaining versioned schemas and compatibility handling.

ADR-011: Risk Prediction as an Independent Service with Hybrid Scoring Strategies

Status: Accepted

Context: FR-07 requires advisory risk prediction. Two distinct clinical approaches are useful: population-level early-warning scores (widely used in hospitals, e.g., NEWS2) that catch clinically dangerous readings for any patient, and patient-specific anomaly detection that catches deviations from an individual's normal baseline. Neither approach alone is sufficient: population thresholds miss patient-specific drift, and personal baselines miss readings that are absolutely dangerous regardless of history. The prediction workload also has different characteristics from ingestion and notification: it consumes events rather than accepting external traffic, it can be scaled independently, and it is expected to evolve as new strategies (ML-based models, ensemble methods) are added.

Decision: Deploy risk prediction as an independent backend service (Prediction Service) alongside Ingestion, Notification, and Escalation Mediator. The service consumes VITAL_RECEIVED events from the message broker, runs two prediction strategies in parallel, and publishes RISK_PREDICTED events back to the broker. Strategies are implemented behind a common interface so that additional approaches can be introduced without modifying existing code. The prototype implements two strategies: Early Warning Score (EWS): a NEWS2-inspired

rule-based scorer that assigns points per vital range (heart rate, SpO2, temperature, respiratory rate, systolic BP) and aggregates to an overall severity score. Personalized Baseline: maintains a rolling window of recent readings per patient, computes mean and standard deviation, and triggers when a new reading's z-score exceeds a configurable threshold (default 2.5σ). Strategies run independently; their outputs are combined into a single prediction event with both scores and a unified severity recommendation.Notification service subscribes to RISK_PREDICTED events and presents high-risk predictions as advisory signals. Based on configured policies, high or critical prediction outcomes may trigger supplementary alert records, complementing threshold-based alerts rather than replacing them.

Consequences: Prediction is decoupled from ingestion, can be scaled and deployed independently, and can evolve (new strategies, trained models, different clinical scoring systems) without changes to other services. The EWS strategy is clinically credible because NEWS2 is the de facto standard in many hospitals. The personalized baseline strategy requires a warmup period per patient before it produces meaningful output; during warmup, EWS alone is used. The service maintains its own per-patient baseline state, which must be persisted for warm restarts. The addition of these services increases operational complexity (additional containers, healthchecks, and consumer connections to the broker) but is justified by the architectural benefit of isolating evolvable prediction logic.

ADR-012: Use Escalation Mediator for Stateful Critical-Alert Workflows

Status: Accepted

Context:
Critical alerts require timed, multi-step escalation when acknowledgments do not occur within expected windows. Encoding this workflow in independent consumers would duplicate state-machine logic and increase coupling between notification paths.

Decision:
Introduce an Escalation Mediator service that consumes ALERT_CREATED events, starts escalation workflows for critical alerts, waits for acknowledgment windows, and escalates recipients in sequence (primary doctor -> on-call doctor -> caregiver/family -> admin). Each escalation step is recorded in the audit trail, and workflow state is persisted for reliability across restarts.

Consequences:
This isolates stateful escalation logic into a dedicated service, improves clinical safety for unacknowledged critical alerts, and strengthens process/runtime architecture. It adds operational complexity due to an additional backend service and workflow state management.

ADR-013: Integrate Chatbot Directly in Notification Service (Prototype Scope)

Status: Accepted

Context:
The prototype requires chatbot-based triage functionality without introducing an additional deployable service. The chatbot shares infrastructure concerns with the Notification Service, including API handling, audit logging, and integration with backend workflows.

Decision:
Implement the chatbot as a direct API capability within the Notification Service (/v1/chatbot/message) instead of as a separate microservice. The chatbot uses a provider strategy order (OpenAI → Gemini → deterministic local rules fallback) to ensure resilience. Chatbot outputs are advisory-only and do not create alerts or trigger notification or escalation workflows.

Consequences:
This reduces deployment complexity and accelerates prototype delivery. It introduces limited coupling with the Notification Service but preserves a clear migration path toward a standalone chatbot service in production. Keeping chatbot outputs advisory avoids unintended clinical side effects and maintains separation from alerting and escalation workflows.

Task 3: Architectural Tactics and Patterns

3.1 Architectural Tactics

AT-01 — Authenticate Users *(Security: Resisting Attacks)* The system verifies user identity before granting access to protected resources. Implemented via JWT-based authentication at the `/auth/token` endpoint, with bcrypt-verified credentials. Addresses: Security (NFR-03), FR-09. Rationale: Ensures only legitimate users can access sensitive healthcare APIs, which is fundamental to protecting patient data.

AT-02 — Authorize Users *(Security: Resisting Attacks)* Access to operations and resources is governed by role-based permissions for ADMIN, DOCTOR, PATIENT, CAREGIVER & SIMULATOR roles, enforced at each protected endpoint. Addresses: Security (NFR-03), Privacy (NFR-04), FR-09. Rationale: Enforces least-privilege access so that each user role sees only what is appropriate for clinical or operational responsibilities.

AT-03 — Maintain Data Confidentiality *(Security: Resisting Attacks)* Sensitive data is protected in transit and at rest. Passwords are stored using bcrypt hashing; transport uses TLS 1.2 or higher in production deployments. Addresses: Security (NFR-03), Privacy (NFR-04). Rationale: Healthcare data

is regulated and highly sensitive; confidentiality controls reduce exposure risk across interfaces and storage.

AT-04 — Identification and Audit *(Security: Detection and Recovery)* All sensitive operations (authentication events, alert creation, acknowledgments, configuration changes) are recorded in the `audit_logs` table with actor, action, target, and metadata fields. Addresses: Privacy (NFR-04), FR-10. Rationale: Enables forensic review, supports HIPAA-aligned auditability, and deters misuse by creating non-repudiable records.

AT-05 — Fault Recovery via D-L Handling and Outbox Retry (Availability: Fault Recovery)

Durable message queues persist alert events across broker restarts. Consumer-side processing failures result in messages being rejected without requeue and routed to a d-letter queue (DLQ) for isolation and later inspection. Separately, the transactional outbox mechanism ensures reliable event publishing with retry handling at the producer side.

Addresses: Availability (NFR-02).

Rationale: Prevents infinite retry loops caused by poison messages, ensures failed messages do not block queue processing, and preserves failed events for debugging or controlled reprocessing while maintaining reliable delivery from the ingestion path.

AT-06 Ping/Echo (Availability: Fault Detection) Docker-level healthchecks are configured for the message broker and the database container. The notification service exposes a `/v1/queue/health` endpoint that queries broker connectivity and returns queue depth for the alerts queue. These health signals allow operators and container orchestration to detect component failure. Addresses: Availability (NFR-02). Rationale: Early detection of component failure is a prerequisite for recovery and for maintaining availability targets.

AT-07 — Use an Intermediary *(Modifiability: Prevent Ripple Effects)* A message broker (RabbitMQ) sits between vital-data ingestion and downstream consumers such as alert processing, notification delivery, and audit logging. Producers have no knowledge of consumers. Addresses: Modifiability (NFR-06). Rationale: New consumers — for example, an analytics pipeline or an additional notification channel — can be added without modifying the ingestion path.

AT-08 — Introduce Concurrency *(Performance: Resource Management)* A background consumer thread processes alert events asynchronously, and WebSocket broadcasts run off the request path so that ingestion requests return quickly. Addresses: Performance (NFR-01), Scalability (NFR-05). Rationale: Removes slow work (notifications, dashboard fan-out) from the synchronous request path, keeping p95 latency low under load.

AT-09 — Built-in Monitors *(Testability: Internal Monitoring)* Structured audit events and the queue-health endpoint expose internal state suitable for automated assertions and operational dashboards. Addresses: Testability (NFR-07), Availability (NFR-02). Rationale: Observable internal state is necessary for fault diagnosis and for writing integration tests that verify cross-subsystem behavior.

AT-10 Pluggable Prediction Strategy (Modifiability: Defer Binding Time) The Prediction service invokes risk scoring through a strategy interface. The prototype implements Early Warning Score and Personalized Baseline strategies; additional strategies (trained ML models, ensembles, specialty-specific scorers) can be added as new implementations without modifying the service orchestration, event schema, or consumer services. Addresses: Modifiability (NFR-06), ASR-05. Rationale: Prediction logic is expected to evolve independently of the rest of the system; isolating it behind a strategy interface limits blast radius when prediction approaches change.

AT-11 Transactional Outbox for Guaranteed Event Delivery (Availability: Fault Recovery) Domain events are persisted in the outbox_events table in the same transaction as the originating state change. A background publisher relays rows to the broker, retrying on failure and marking them PUBLISHED only after broker acknowledgment. This decouples event delivery from broker availability at the moment of the transaction. Addresses: Availability (NFR-02). Rationale: Eliminates the dual-write race where a DB commit could succeed but broker publish could fail, causing a committed alert with no event ever emitted. Guarantees at-least-once delivery even across broker outages.

AT-12 Transactions (Availability: Fault Prevention) Related changes to persistent state (alert creation, outbox enqueue, audit log) are performed within a single database transaction. The transaction either commits all changes atomically or rolls back all of them, preventing partial state that could cause downstream consumers to see inconsistent data. Addresses: Availability (NFR-02), Privacy (NFR-04). Rationale: Atomic commits are the foundation of the transactional outbox pattern and ensure audit records cannot diverge from the alerts they describe.

AT-13 Exception Handling with Reconnection (Availability: Fault Recovery) The broker consumer uses an exception-handling loop with bounded-backoff reconnection: if the broker connection drops, the consumer logs the failure and attempts to reconnect after a short delay. This allows the consumer to recover automatically from transient broker outages without operator intervention. Addresses: Availability (NFR-02). Rationale: Transient broker disconnects are a normal operational event; automatic reconnection prevents a single disconnect from requiring a consumer restart.

3.2 Implementation Patterns

The architecture uses the following implementation patterns.

DP-01 Publish–Subscribe Pattern (Event-Driven Inter-Service Communication)
Vital-sign and alert domain events are exchanged asynchronously via RabbitMQ. Ingestion publishes VITAL_RECEIVED and ALERT_CREATED; Prediction consumes VITAL_RECEIVED and publishes RISK_PREDICTED; Notification consumes ALERT_CREATED and RISK_PREDICTED; Escalation Mediator consumes ALERT_CREATED events for escalation workflows.
Role in architecture: Decouples producers from consumers and enables independent service evolution and deployment.
Why selected: New consumers can subscribe without modifying existing producers.
NFR impact: Improves Modifiability (NFR-06), supports Scalability and Performance (NFR-05, NFR-01), and improves Availability via loose coupling (NFR-02).

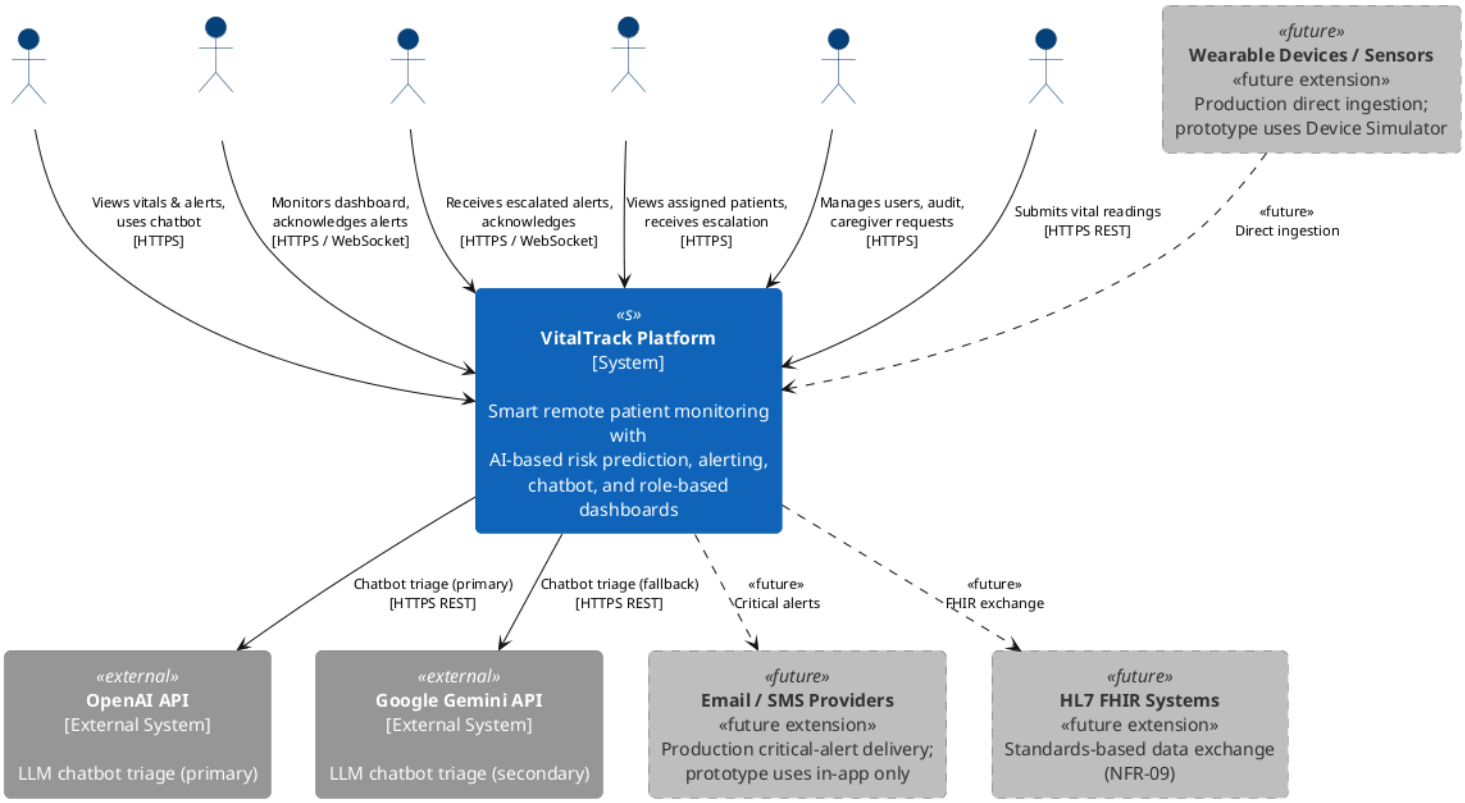
DP-02 Strategy Pattern (Pluggable Prediction and Chatbot Triage Logic)
Variation points are modeled as strategies. Prediction uses multiple risk strategies (NEWS2-inspired Early Warning Score and personalized baseline), and chatbot triage uses provider strategies (OpenAI / Gemini → local rules fallback).
Role in architecture: Supports interchangeable algorithms and providers under stable orchestration.
Why selected: Prediction logic and AI provider integrations are expected to evolve independently.
NFR impact: Improves Modifiability and Maintainability (NFR-06), supports AI evolution (ASR-05), and improves Testability (NFR-07).

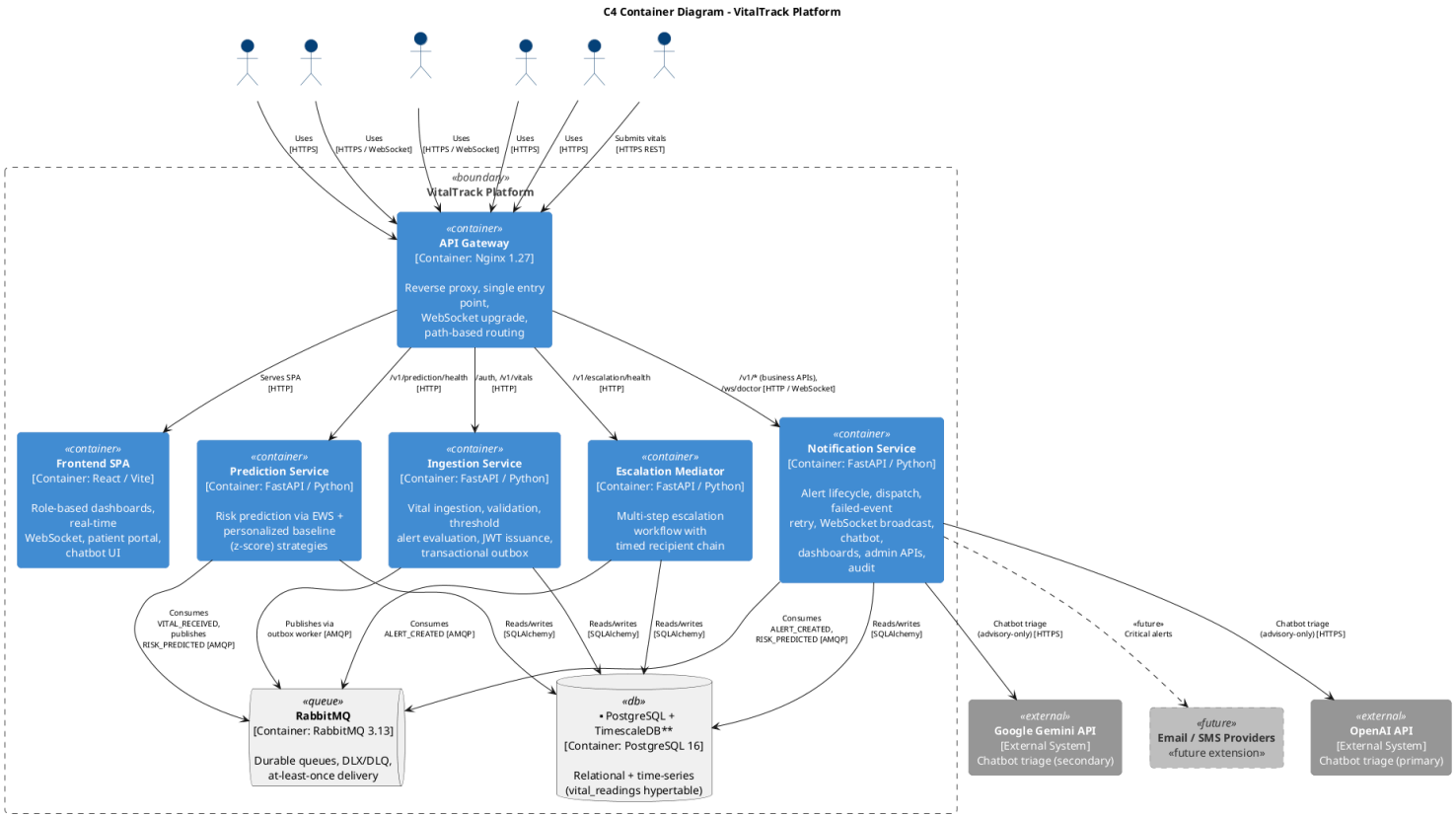
DP-03 Adapter Pattern (Normalization of External and Provider Responses)
Adapters normalize heterogeneous provider outputs into a canonical internal contract (for example, chatbot provider output normalized to {risk_level, reply}).
Role in architecture: Shields business flow from provider-specific payload formats.
Why selected: Reduces ripple effects when integrating or changing external providers.
NFR impact: Improves Interoperability (NFR-09) and Modifiability (NFR-06).

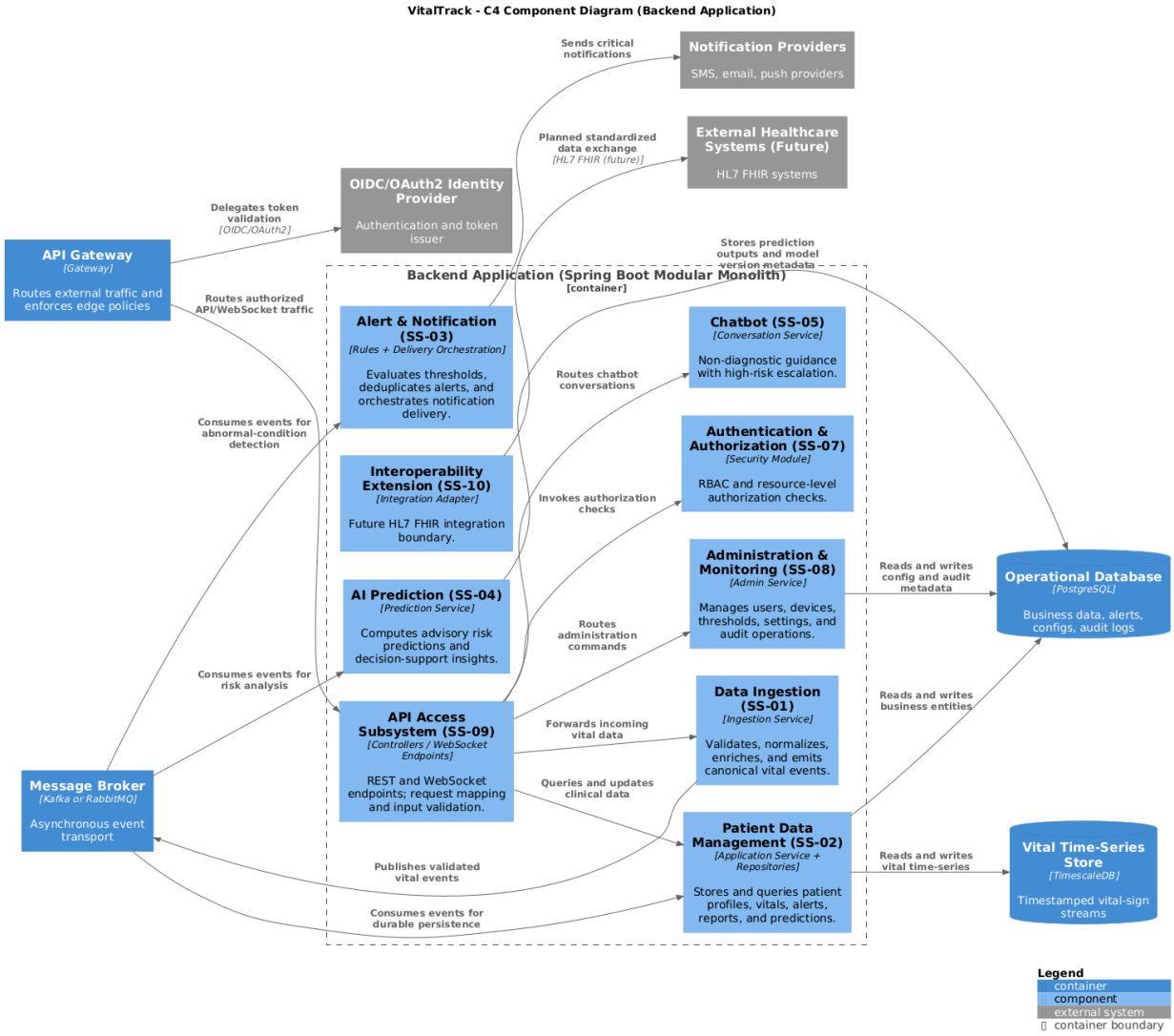
DP-04 Factory Method Pattern (Centralized Chatbot Strategy Construction)
A chatbot triage factory (ChatbotTriageFactory) builds strategy instances and shared adapters, and defines the execution order (OpenAI → Gemini → local rules fallback).
Role in architecture: Centralizes creation logic and keeps endpoint orchestration free from construction branching.
Why selected: Makes provider composition and execution order configurable and extensible with minimal caller changes.
NFR impact: Improves Modifiability and Maintainability (NFR-06) and supports clean extension for future chatbot providers.

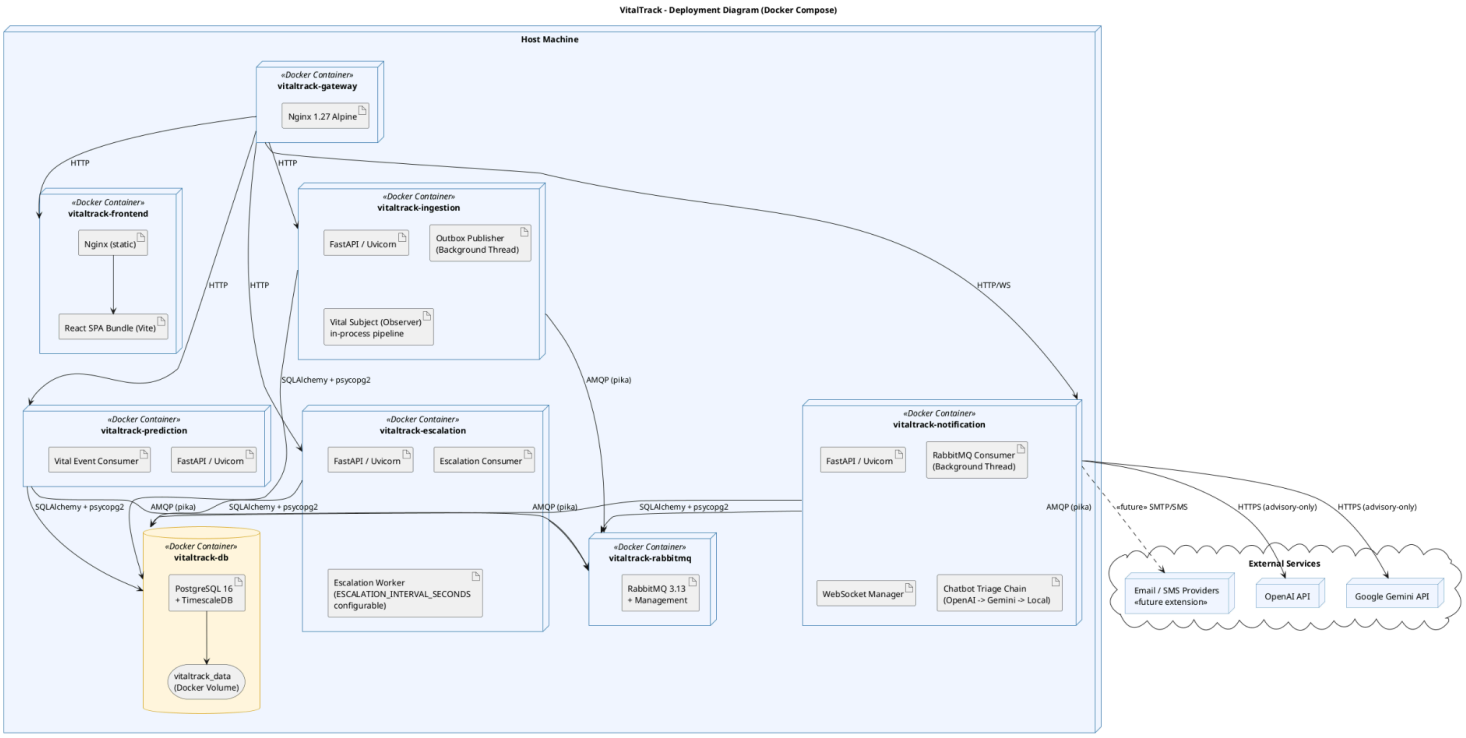
DP-05 Observer Pattern (In-Process Vital Processing Pipeline)
Within the Ingestion service, vital processing uses an Observer structure (VitalSubject with observers such as risk-score and alert-evaluation observers). A shared observation context is passed through observers, and each observer contributes derived results without tightly coupling all logic in one function.
Role in architecture: Enables staged, extensible in-process processing for each ingested vital event.
Why selected: New processing steps can be added as observers with minimal impact on existing logic.
NFR impact: Improves Modifiability and Maintainability (NFR-06) and improves Testability (NFR-07).

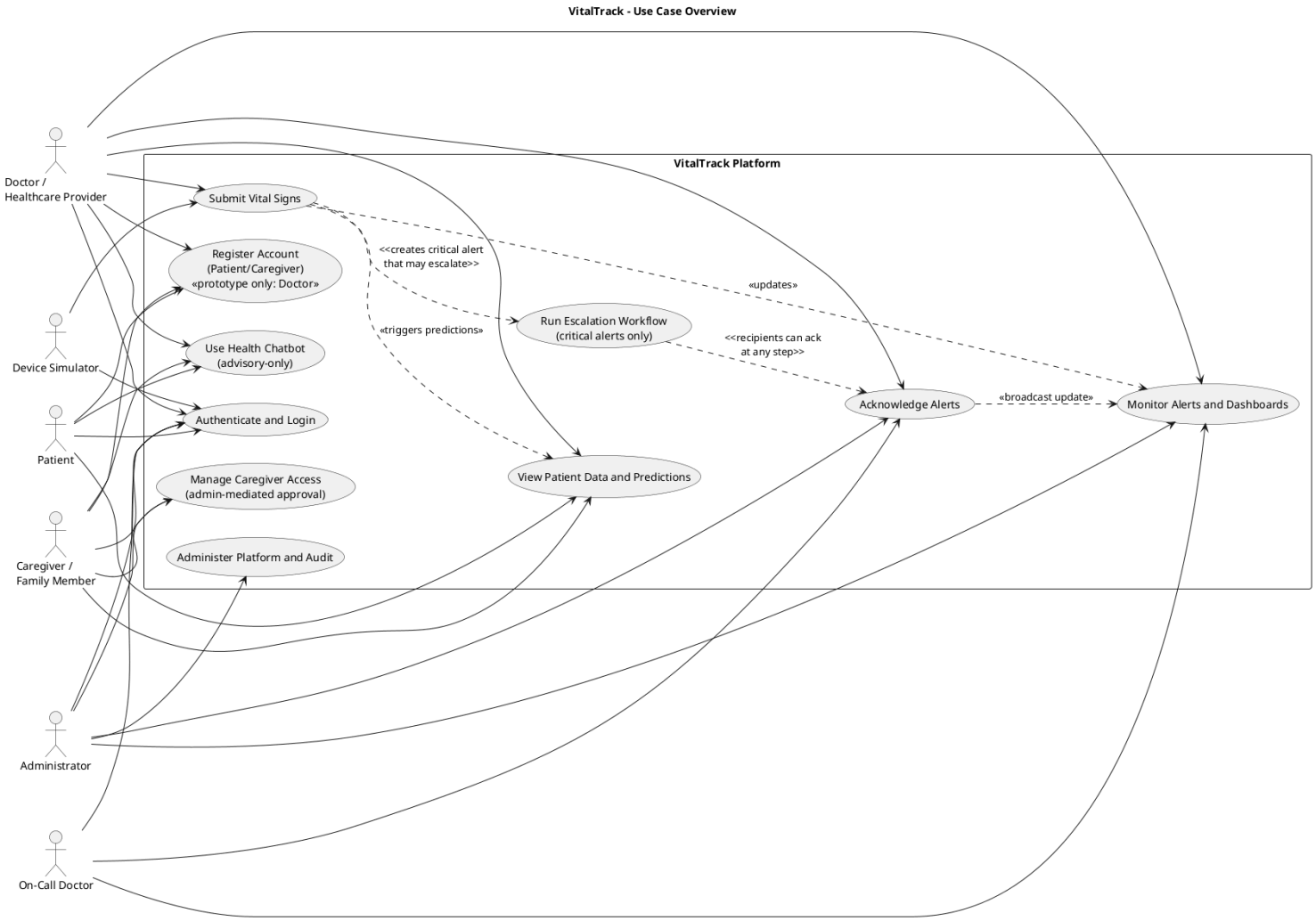
C4 Context Diagram - VitalTrack Remote Patient Monitoring Platform

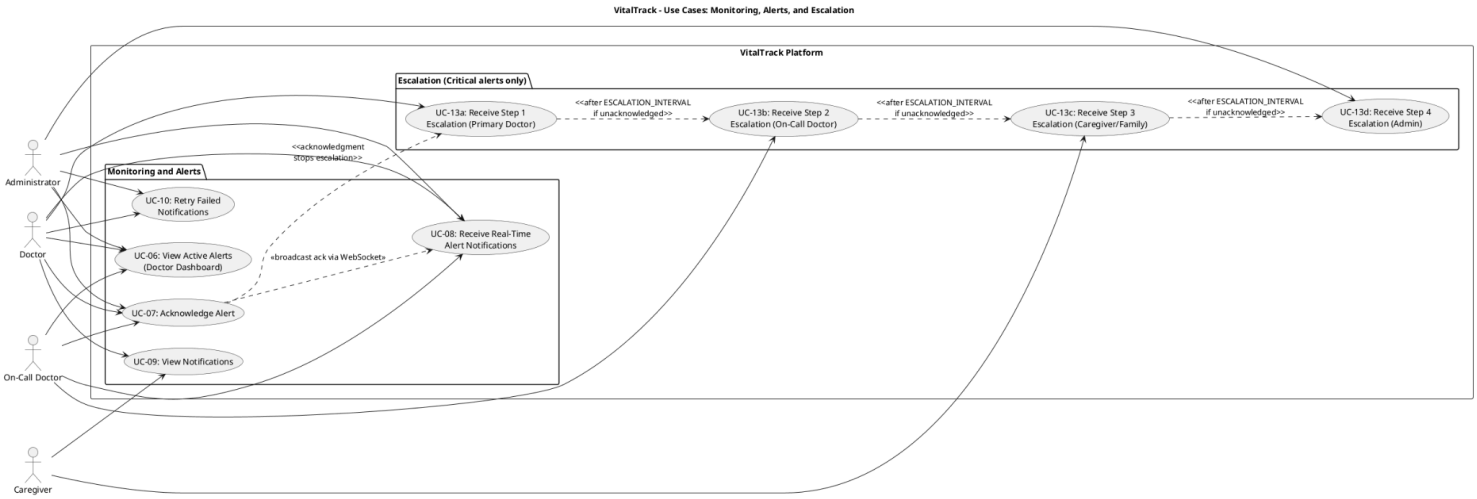


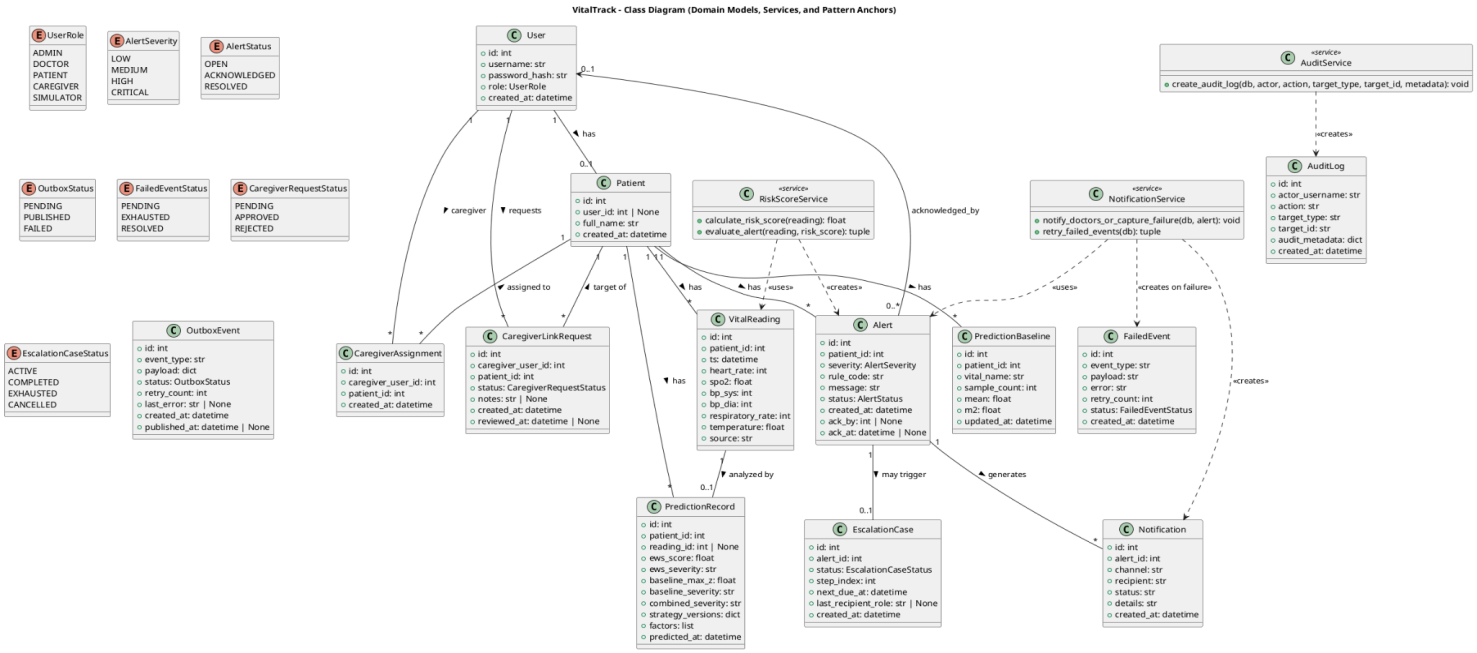

C4 Container Diagram - VitalTrack Platform
Uses [HTTPS]
Uses [HTTPS / WebSocket]
Uses [HTTPS / WebSocket]
Uses [HTTPS]
Uses [HTTPS]
Submits vitals [HTTPS REST]
«boundary»
VitalTrack Platform
«container»
API Gateway
[Container: Nginx 1.27]
Reverse proxy, single entry point,
WebSocket upgrade,
path-based routing
Serves SPA [HTTP]
/v1/prediction/health [HTTP]
/auth, /v1/vitals [HTTP]
/v1/escalation/health [HTTP]
/v1/* (business APIs), /ws/doctor [HTTP / WebSocket]
«container»
Frontend SPA
[Container: React / Vite]
Role-based dashboards, real-time
WebSocket, patient portal, chatbot UI
«container»
Prediction Service
[Container: FastAPI / Python]
Risk prediction via EWS + personalized baseline
(z-score) strategies
«container»
Ingestion Service
[Container: FastAPI / Python]
Vital ingestion, validation, threshold
alert evaluation, JWT issuance, transactional outbox
«container»
Escalation Mediator
[Container: FastAPI / Python]
Multi-step escalation workflow with
timed recipient chain
«container»
Notification Service
[Container: FastAPI / Python]
Alert lifecycle, dispatch, failed-event
retry, WebSocket broadcast, chatbot,
dashboards, admin APIs, audit
Consumes VITAL_RECEIVED, publishes RISK_PREDICTED [AMQP]
Publishes via outbox worker [AMQP]
Consumes ALERT_CREATED [AMQP]
Reads/writes [SQLAlchemy]
Reads/writes [SQLAlchemy]
Reads/writes [SQLAlchemy]
Consumes ALERT_CREATED, RISK_PREDICTED [AMQP]
Reads/writes [SQLAlchemy]
Chatbot triage (advisory-only) [HTTPS]
«future» Critical alerts
Chatbot triage (advisory-only) [HTTPS]
«queue»
RabbitMQ
[Container: RabbitMQ 3.13]
Durable queues, DLX/DLQ, at-least-once delivery
«db»
• PostgreSQL + TimescaleDB**
[Container: PostgreSQL 16]
Relational + time-series
(vital_readings hypertable)
«external»
Google Gemini API
[External System]
Chatbot triage (secondary)
«future»
Email / SMS Providers
«future extension»
«external»
OpenAI API
[External System]
Chatbot triage (primary)


VitalTrack - C4 Component Diagram (Backend Application)
Notification Providers
SMS, email, push providers
Sends critical notifications
External Healthcare Systems (Future)
HL7 FHIR systems
Planned standardized data exchange
[HL7 FHIR (future)]
OIDC/OAuth2 Identity Provider
Authentication and token issuer
Delegates token validation
[OIDC/OAuth2]
API Gateway
[Gateway]
Routes external traffic and enforces edge policies
Routes authorized API/WebSocket traffic
Stores prediction outputs and model version metadata
Backend Application (Spring Boot Modular Monolith)
[container]
Alert & Notification (SS-03)
[Rules + Delivery Orchestration]
Evaluates thresholds, deduplicates alerts, and orchestrates notification delivery.
Chatbot (SS-05)
[Conversation Service]
Non-diagnostic guidance with high-risk escalation.
Routes chatbot conversations
Consumes events for abnormal-condition detection
Authentication & Authorization (SS-07)
[Security Module]
RBAC and resource-level authorization checks.
Interoperability Extension (SS-10)
[Integration Adapter]
Future HL7 FHIR integration boundary.
Invokes authorization checks
Administration & Monitoring (SS-08)
[Admin Service]
Manages users, devices, thresholds, settings, and audit operations.
Reads and writes config and audit metadata
Operational Database
[PostgreSQL]
Business data, alerts, configs, audit logs
AI Prediction (SS-04)
[Prediction Service]
Computes advisory risk predictions and decision-support insights.
Routes administration commands
Consumes events for risk analysis
Data Ingestion (SS-01)
[Ingestion Service]
Validates, normalizes, enriches, and emits canonical vital events.
Reads and writes business entities
API Access Subsystem (SS-09)
[Controllers / WebSocket Endpoints]
REST and WebSocket endpoints; request mapping and input validation.
Forwards incoming vital data
Queries and updates clinical data
Message Broker
[Kafka or RabbitMQ]
Asynchronous event transport
Patient Data Management (SS-02)
[Application Service + Repositories]
Stores and queries patient profiles, vitals, alerts, reports, and predictions.
Reads and writes vital time-series
Vital Time-Series Store
[TimescaleDB]
Timestamped vital-sign streams
Publishes validated vital events
Consumes events for durable persistence
Legend
container
component
external system
container boundary

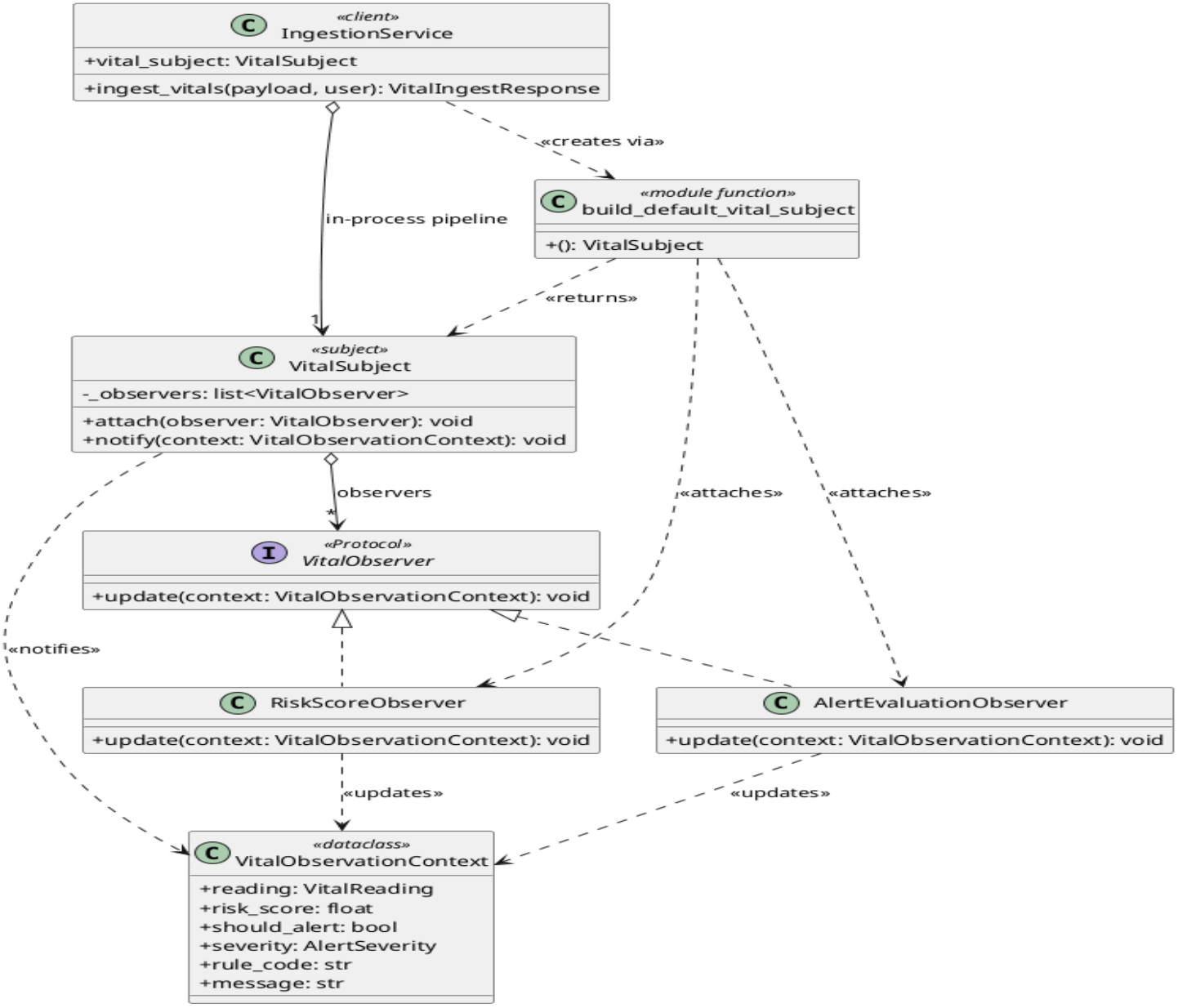

VitalTrack - Deployment Diagram (Docker Compose)
Host Machine
«Docker Container»
vitaltrack-gateway
Nginx 1.27 Alpine
«Docker Container»
vitaltrack-frontend
Nginx (static)
React SPA Bundle (Vite)
«Docker Container»
vitaltrack-ingestion
FastAPI / Uvicorn
Outbox Publisher
(Background Thread)
Vital Subject (Observer)
in-process pipeline
«Docker Container»
vitaltrack-prediction
Vital Event Consumer
FastAPI / Uvicorn
«Docker Container»
vitaltrack-escalation
FastAPI / Uvicorn
Escalation Consumer
Escalation Worker
(ESCALATION_INTERVAL_SECONDS
configurable)
«Docker Container»
vitaltrack-notification
FastAPI / Uvicorn
RabbitMQ Consumer
(Background Thread)
WebSocket Manager
Chatbot Triage Chain
(OpenAI -> Gemini -> Local)
«Docker Container»
vitaltrack-rabbitmq
RabbitMQ 3.13
+ Management
«Docker Container»
vitaltrack-db
PostgreSQL 16
+ TimescaleDB
vitaltrack_data
(Docker Volume)
HTTP
HTTP
HTTP
HTTP
HTTP/WS
SQLAlchemy + psycopg2
AMQP (pika)
SQLAlchemy + psycopg2
AMQP (pika)
SQLAlchemy + psycopg2
AMQP (pika)
SQLAlchemy + psycopg2
AMQP (pika)
«future» SMTP/SMS
HTTPS (advisory-only)
HTTPS (advisory-only)
External Services
Email / SMS Providers
«future extension»
OpenAI API
Google Gemini API

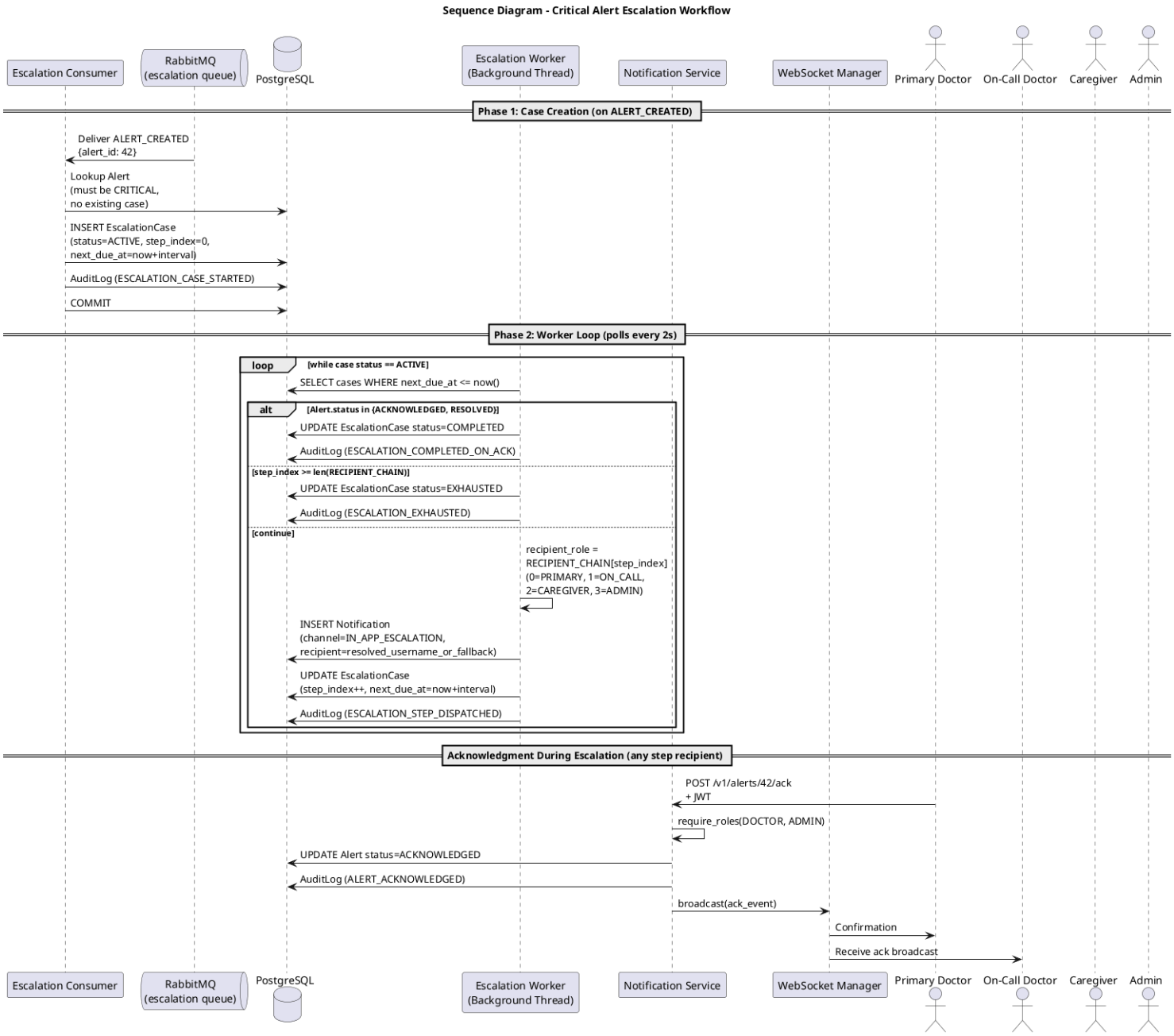

VitalTrack - Use Case Overview
VitalTrack Platform
Doctor / Healthcare Provider
Device Simulator
Patient
Caregiver / Family Member
Administrator
On-Call Doctor
Submit Vital Signs
Register Account (Patient/Caregiver) «prototype only: Doctor»
Use Health Chatbot (advisory-only)
Authenticate and Login
Manage Caregiver Access (admin-mediated approval)
Administer Platform and Audit
Run Escalation Workflow (critical alerts only)
View Patient Data and Predictions
Acknowledge Alerts
Monitor Alerts and Dashboards
<<creates critical alert that may escalate>>
«triggers predictions»
«updates»
<<recipients can ack at any step>>
«broadcast update»

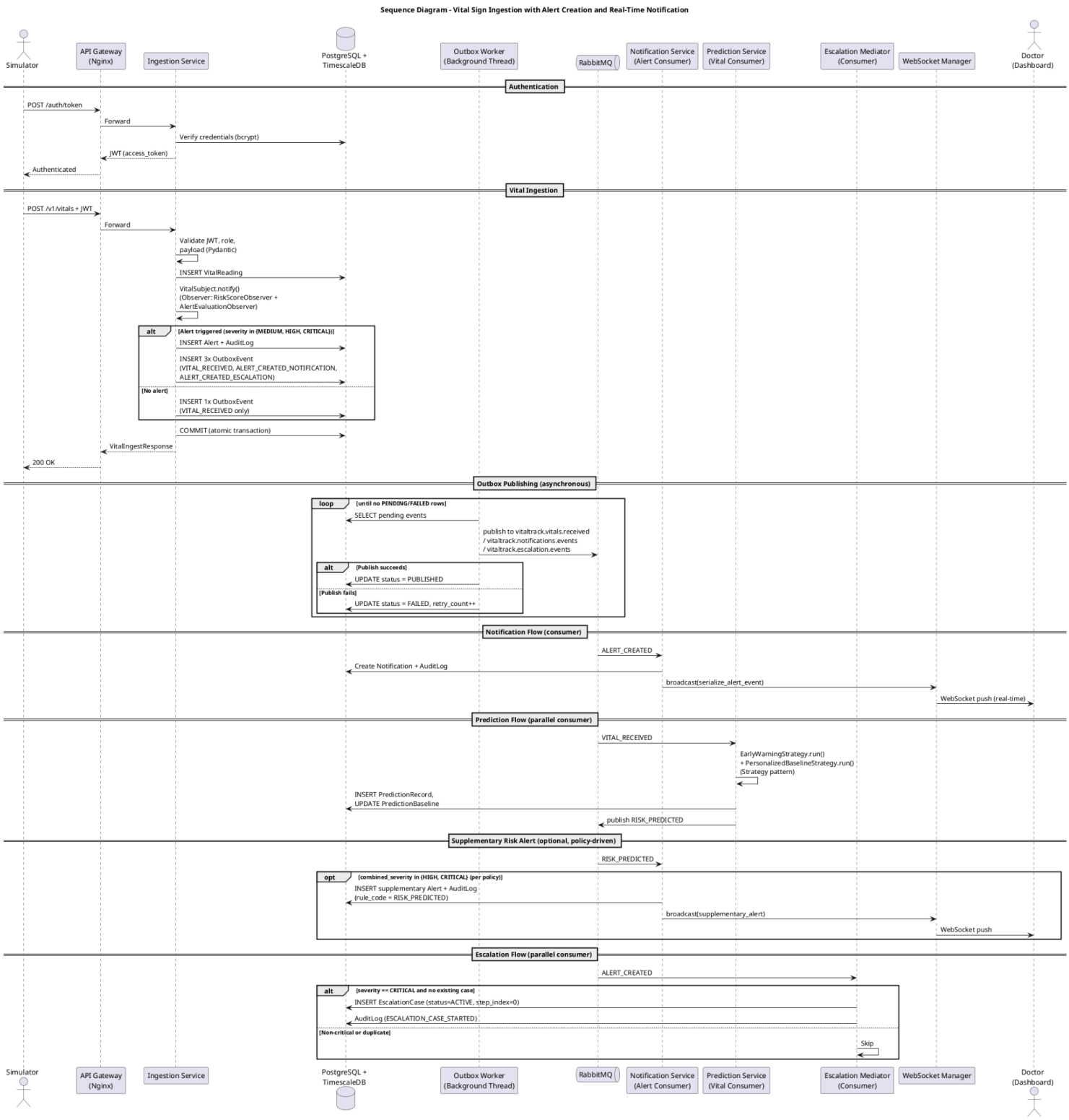

VitalTrack - Use Cases: Monitoring, Alerts, and Escalation
VitalTrack Platform
Escalation (Critical alerts only)
UC-13a: Receive Step 1 Escalation (Primary Doctor)
<<after ESCALATION_INTERVAL if unacknowledged>>
UC-13b: Receive Step 2 Escalation (On-Call Doctor)
<<after ESCALATION_INTERVAL if unacknowledged>>
UC-13c: Receive Step 3 Escalation (Caregiver/Family)
<<after ESCALATION_INTERVAL if unacknowledged>>
UC-13d: Receive Step 4 Escalation (Admin)
Monitoring and Alerts
UC-10: Retry Failed Notifications
UC-06: View Active Alerts (Doctor Dashboard)
UC-07: Acknowledge Alert
UC-09: View Notifications
UC-08: Receive Real-Time Alert Notifications
<<acknowledgment stops escalation>>
«broadcast ack via WebSocket»
Administrator
Doctor
On-Call Doctor
Caregiver


VitalTrack - Class Diagram (Domain Models, Services, and Pattern Anchors)
UserRole
ADMIN
DOCTOR
PATIENT
CAREGIVER
SIMULATOR
AlertSeverity
LOW
MEDIUM
HIGH
CRITICAL
AlertStatus
OPEN
ACKNOWLEDGED
RESOLVED
OutboxStatus
PENDING
PUBLISHED
FAILED
FailedEventStatus
PENDING
EXHAUSTED
RESOLVED
CaregiverRequestStatus
PENDING
APPROVED
REJECTED
EscalationCaseStatus
ACTIVE
COMPLETED
EXHAUSTED
CANCELLED
User
+ id: int
+ username: str
+ password_hash: str
+ role: UserRole
+ created_at: datetime
«service»
AuditService
+ create_audit_log(db, actor, action, target_type, target_id, metadata): void
«creates»
AuditLog
+ id: int
+ actor_username: str
+ action: str
+ target_type: str
+ target_id: str
+ audit_metadata: dict
+ created_at: datetime
Patient
+ id: int
+ user_id: int | None
+ full_name: str
+ created_at: datetime
«service»
RiskScoreService
+ calculate_risk_score(reading): float
+ evaluate_alert(reading, risk_score): tuple
«service»
NotificationService
+ notify_doctors_or_capture_failure(db, alert): void
+ retry_failed_events(db): tuple
OutboxEvent
+ id: int
+ event_type: str
+ payload: dict
+ status: OutboxStatus
+ retry_count: int
+ last_error: str | None
+ created_at: datetime
+ published_at: datetime | None
CaregiverAssignment
+ id: int
+ caregiver_user_id: int
+ patient_id: int
+ created_at: datetime
CaregiverLinkRequest
+ id: int
+ caregiver_user_id: int
+ patient_id: int
+ status: CaregiverRequestStatus
+ notes: str | None
+ created_at: datetime
+ reviewed_at: datetime | None
VitalReading
+ id: int
+ patient_id: int
+ ts: datetime
+ heart_rate: int
+ spo2: float
+ bp_sys: int
+ bp_dia: int
+ respiratory_rate: int
+ temperature: float
+ source: str
Alert
+ id: int
+ patient_id: int
+ severity: AlertSeverity
+ rule_code: str
+ message: str
+ status: AlertStatus
+ created_at: datetime
+ ack_by: int | None
+ ack_at: datetime | None
PredictionBaseline
+ id: int
+ patient_id: int
+ vital_name: str
+ sample_count: int
+ mean: float
+ m2: float
+ updated_at: datetime
FailedEvent
+ id: int
+ event_type: str
+ payload: str
+ error: str
+ retry_count: int
+ status: FailedEventStatus
+ created_at: datetime
PredictionRecord
+ id: int
+ patient_id: int
+ reading_id: int | None
+ ews_score: float
+ ews_severity: str
+ baseline_max_z: float
+ baseline_severity: str
+ combined_severity: str
+ strategy_versions: dict
+ factors: list
+ predicted_at: datetime
EscalationCase
+ id: int
+ alert_id: int
+ status: EscalationCaseStatus
+ step_index: int
+ next_due_at: datetime
+ last_recipient_role: str | None
+ created_at: datetime
Notification
+ id: int
+ alert_id: int
+ channel: str
+ recipient: str
+ status: str
+ details: str
+ created_at: datetime
has
caregiver
requests
acknowledged_by
assigned to
target of
«uses»
«creates»
analyzed by
may trigger
generates
«creates on failure»

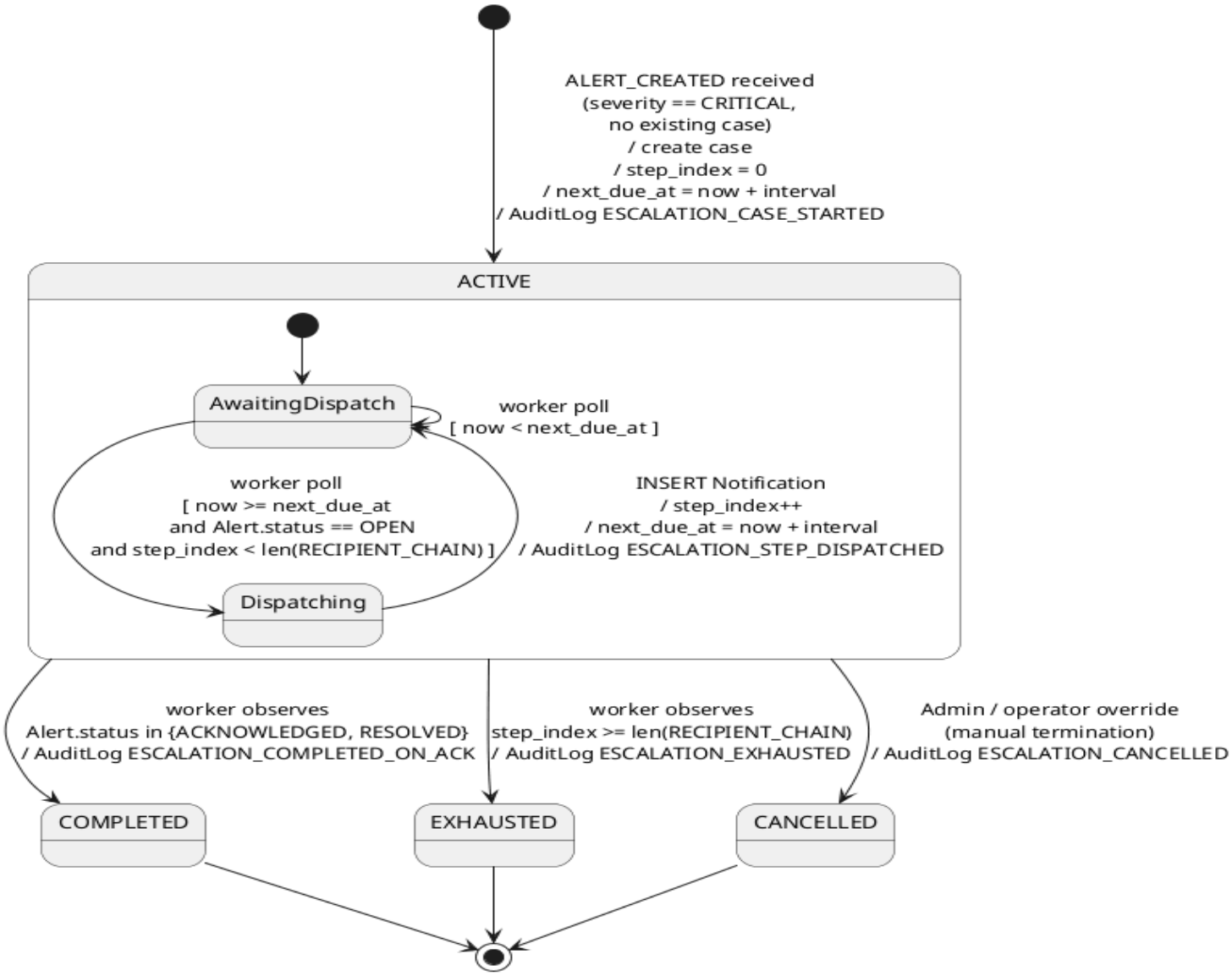
VitalTrack - DP-05 Observer (In-Process Vital Processing)

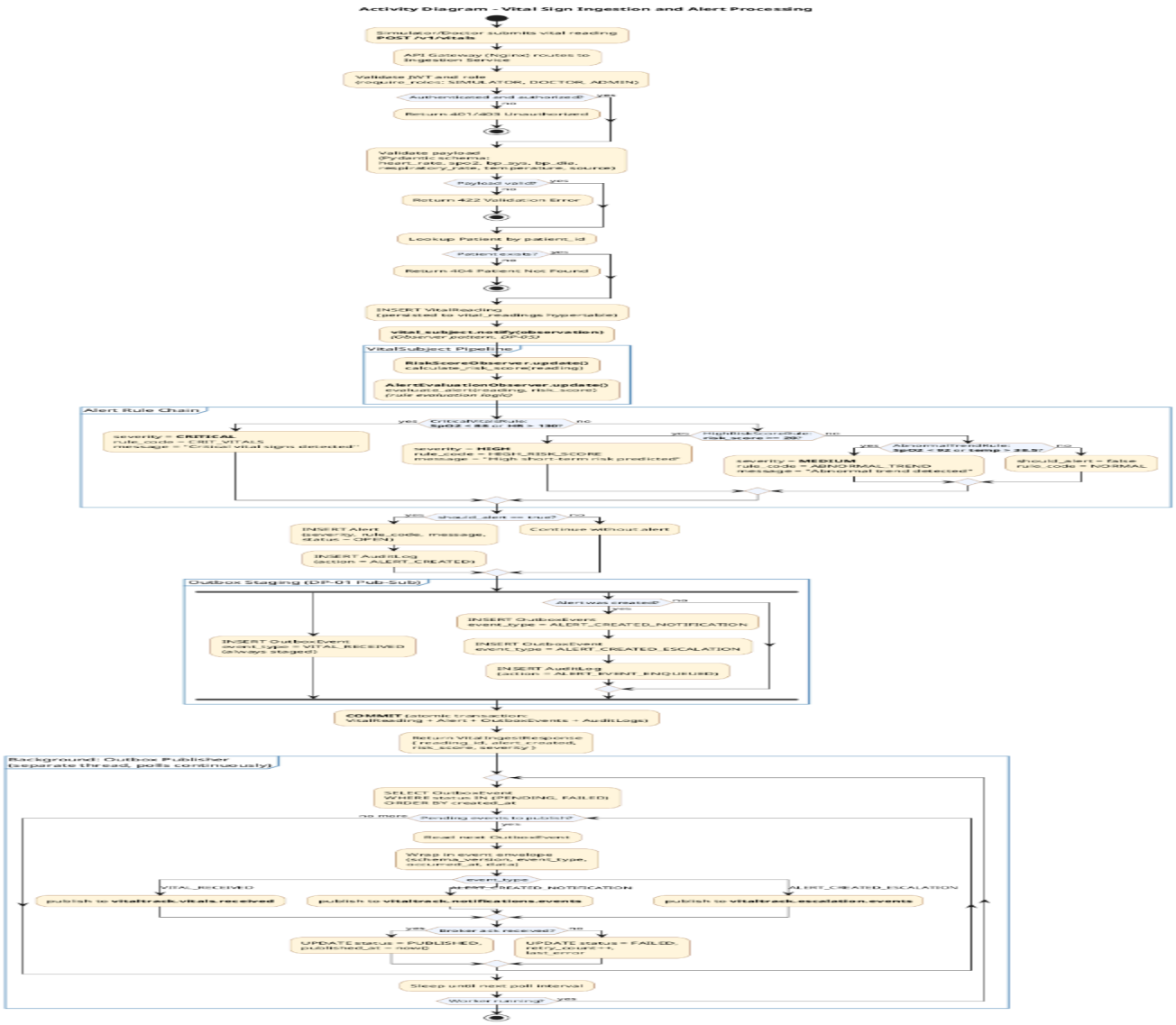



Sequence Diagram - Critical Alert Escalation Workflow
Escalation Consumer
RabbitMQ (escalation queue)
PostgreSQL
Escalation Worker (Background Thread)
Notification Service
WebSocket Manager
Primary Doctor
On-Call Doctor
Caregiver
Admin
Phase 1: Case Creation (on ALERT_CREATED)
Deliver ALERT_CREATED {alert_id: 42}
Lookup Alert (must be CRITICAL, no existing case)
INSERT EscalationCase (status=ACTIVE, step_index=0, next_due_at=now+interval)
AuditLog (ESCALATION_CASE_STARTED)
COMMIT
Phase 2: Worker Loop (polls every 2s)
loop
[while case status == ACTIVE]
SELECT cases WHERE next_due_at <= now()
alt
[Alert.status in {ACKNOWLEDGED, RESOLVED}]
UPDATE EscalationCase status=COMPLETED
AuditLog (ESCALATION_COMPLETED_ON_ACK)
[step_index >= len(RECIPIENT_CHAIN)]
UPDATE EscalationCase status=EXHAUSTED
AuditLog (ESCALATION_EXHAUSTED)
[continue]
recipient_role = RECIPIENT_CHAIN[step_index] (0=PRIMARY, 1=ON_CALL, 2=CAREGIVER, 3=ADMIN)
INSERT Notification (channel=IN_APP_ESCALATION, recipient=resolved_username_or_fallback)
UPDATE EscalationCase (step_index++, next_due_at=now+interval)
AuditLog (ESCALATION_STEP_DISPATCHED)
Acknowledgment During Escalation (any step recipient)
POST /v1/alerts/42/ack + JWT
require_roles(DOCTOR, ADMIN)
UPDATE Alert status=ACKNOWLEDGED
AuditLog (ALERT_ACKNOWLEDGED)
broadcast(ack_event)
Confirmation
Receive ack broadcast

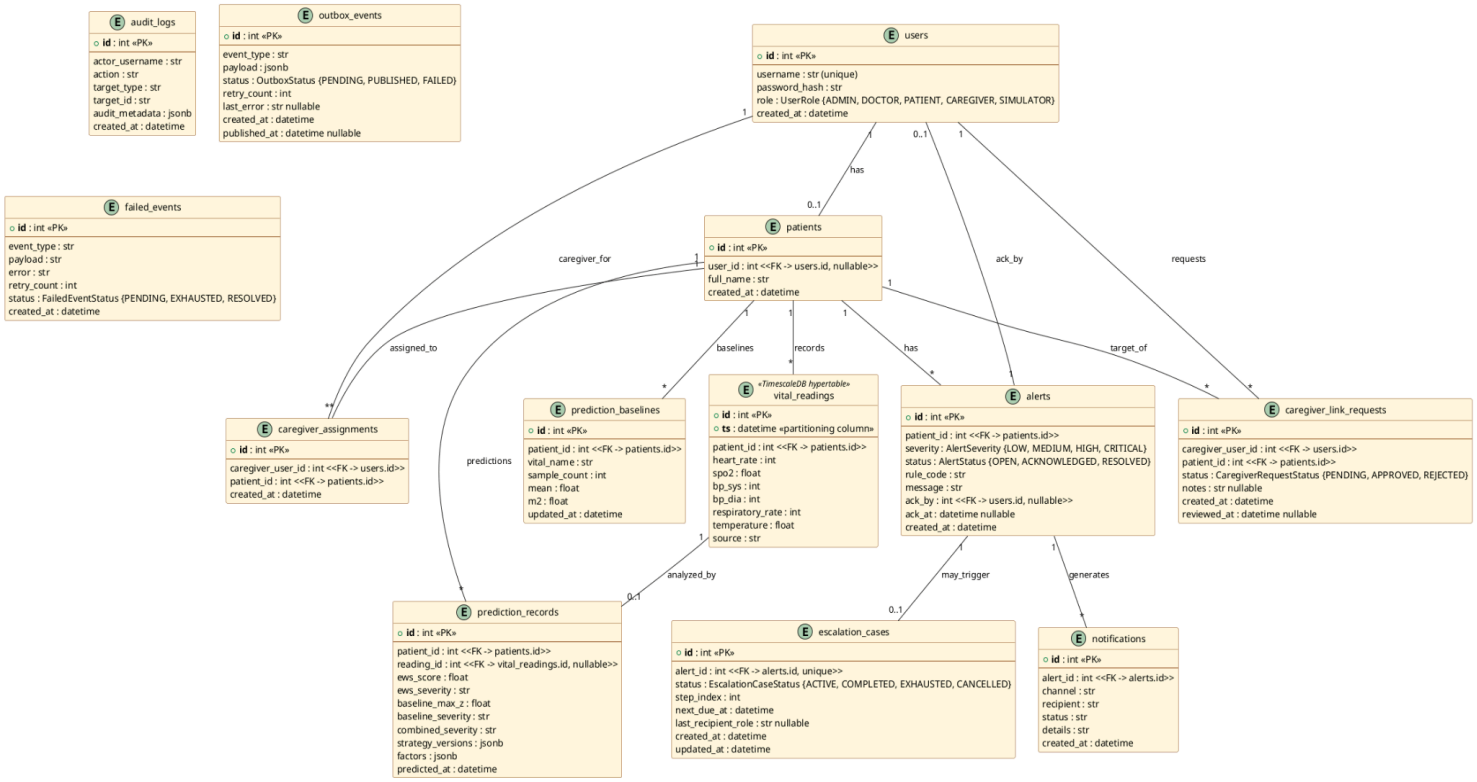

Sequence Diagram - Vital Sign Ingestion with Alert Creation and Real-Time Notification
Simulator
API Gateway (Nginx)
Ingestion Service
PostgreSQL + TimescaleDB
Outbox Worker (Background Thread)
RabbitMQ
Notification Service (Alert Consumer)
Prediction Service (Vital Consumer)
Escalation Mediator (Consumer)
WebSocket Manager
Doctor (Dashboard)
Authentication
POST /auth/token
Forward
Verify credentials (bcrypt)
JWT (access_token)
Authenticated
Vital Ingestion
POST /v1/vitals + JWT
Forward
Validate JWT, role, payload (Pydantic)
INSERT VitalReading
VitalSubject.notify() (Observer: RiskScoreObserver + AlertEvaluationObserver)
alt
[Alert triggered (severity in (MEDIUM, HIGH, CRITICAL))]
INSERT Alert + AuditLog
INSERT 3x OutboxEvent (VITAL_RECEIVED, ALERT_CREATED_NOTIFICATION, ALERT_CREATED_ESCALATION)
[No alert]
INSERT 1x OutboxEvent (VITAL_RECEIVED only)
COMMIT (atomic transaction)
VitalIngestResponse
200 OK
Outbox Publishing (asynchronous)
loop
[until no PENDING/FAILED rows]
SELECT pending events
publish to vitaltrack.vitals.received / vitaltrack.notifications.events / vitaltrack.escalation.events
alt
[Publish succeeds]
UPDATE status = PUBLISHED
[Publish fails]
UPDATE status = FAILED, retry_count++
Notification Flow (consumer)
ALERT_CREATED
Create Notification + AuditLog
broadcast(serialize_alert_event)
WebSocket push (real-time)
Prediction Flow (parallel consumer)
VITAL_RECEIVED
EarlyWarningStrategy.run() + PersonalizedBaselineStrategy.run() (Strategy pattern)
INSERT PredictionRecord, UPDATE PredictionBaseline
publish RISK_PREDICTED
Supplementary Risk Alert (optional, policy-driven)
RISK_PREDICTED
opt
[combined_severity in (HIGH, CRITICAL) (per policy)]
INSERT supplementary Alert + AuditLog (rule_code = RISK_PREDICTED)
broadcast(supplementary_alert)
WebSocket push
Escalation Flow (parallel consumer)
ALERT_CREATED
alt
[severity == CRITICAL and no existing case]
INSERT EscalationCase (status=ACTIVE, step_index=0)
AuditLog (ESCALATION_CASE_STARTED)
[Non-critical or duplicate]
Skip

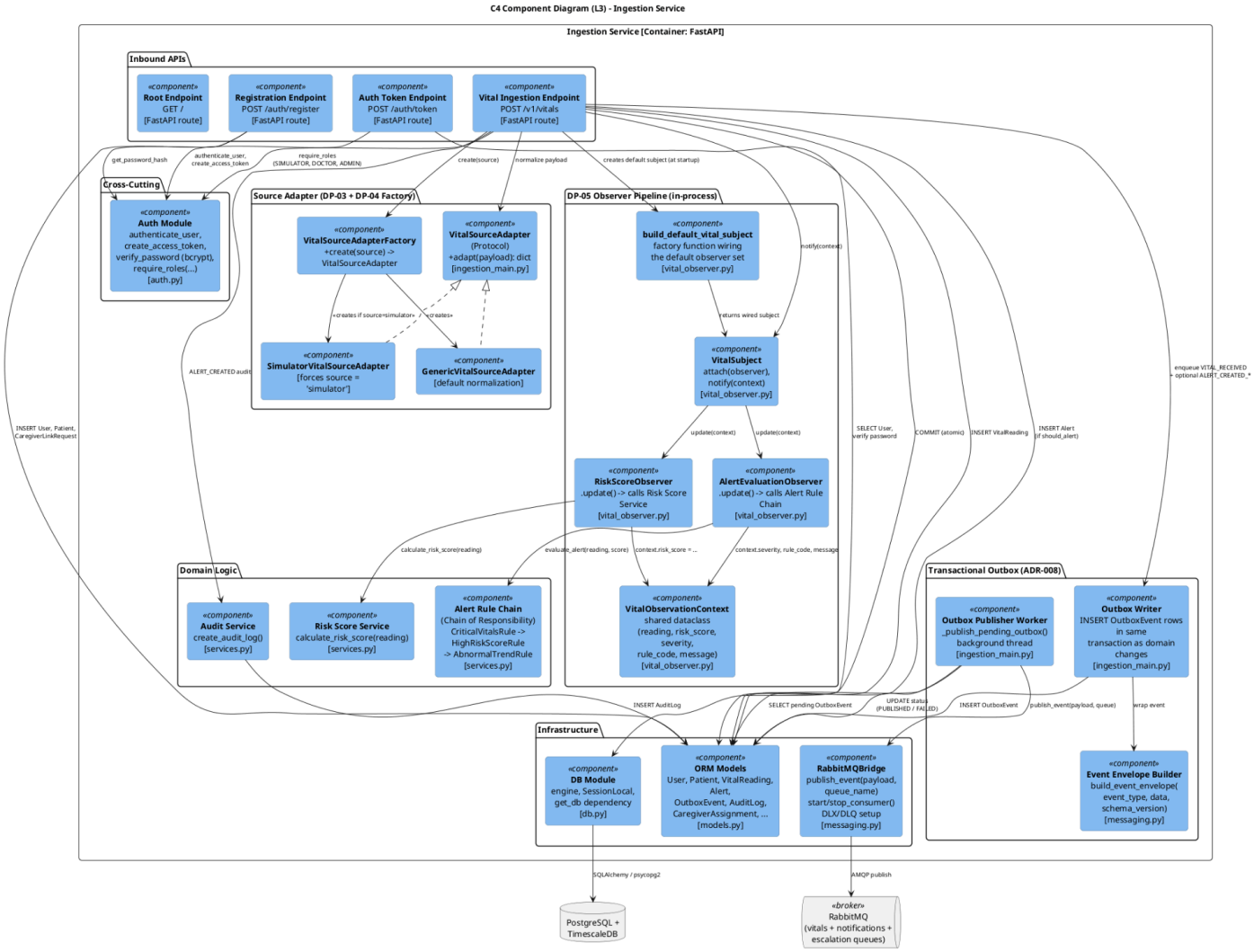
VitalTrack - State Machine: EscalationCase

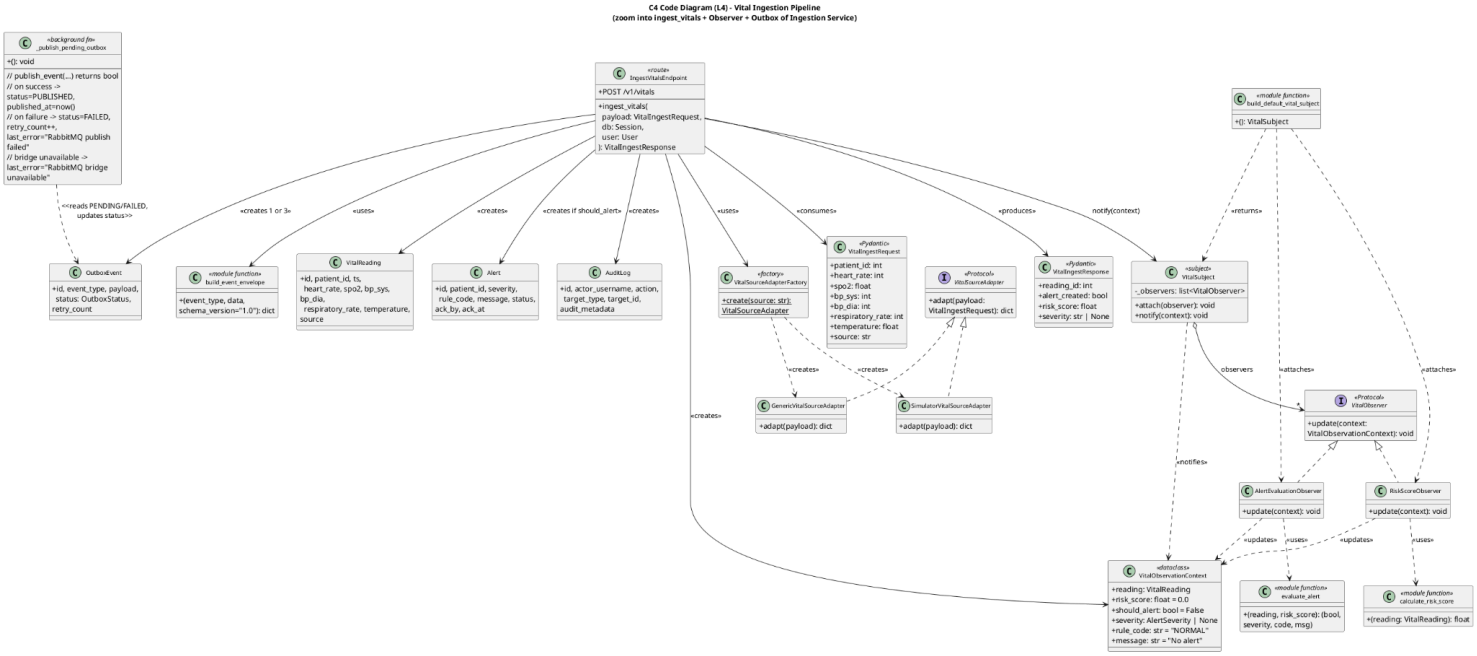



Activity Diagram - Vital Sign Ingestion and Alert Processing
Simulator/Doctor submits vital reading
POST /v1/vitals
API Gateway (Nginx) routes to
Ingestion Service
Validate JWT and role
(require_roles(), SIMULATOR, DOCTOR, ADMIN)
Authenticated and authorized?
yes
no
Return 401/403 Unauthorized
Validate payload
(patient_id, recorded_at,
heart_rate, spo2, bp_sys, bp_dia,
respiratory_rate, temperature, source)
Payload valid?
yes
no
Return 422 Validation Error
Lookup Patient by patient_id
Patient exists?
yes
no
Return 404 Patient Not Found
INSERT VitalReading
(persisted to vital_readings hypertable)
vital_subject.notify(observation)
(Observer pattern, DP-03)
VitalSubject Pipeline
RiskScoreObserver.update()
calculate_risk_score(reading)
AlertEvaluationObserver.update()
evaluate_alert(reading, risk_score)
(rule evaluation logic)
Alert Rule Chain
CriticalRuleRule:
SpO2 < 85 or HR > 130?
yes
no
severity = CRITICAL
rule_code = CRIT_VITALS
message = "Critical vital signs detected"
HighRiskScoreRule:
risk_score >= 80?
yes
no
severity = HIGH
rule_code = HIGH_RISK_SCORE
message = "High short-term risk predicted"
AbnormalTrendRule:
spo2 < 92 or temp > 38.5?
yes
no
severity = MEDIUM
rule_code = ABNORMAL_TREND
message = "Abnormal trend detected"
should_alert = false
rule_code = NORMAL
should_alert == true?
yes
no
INSERT Alert
(severity, rule_code, message,
status = OPEN)
INSERT AuditLog
(action = ALERT_CREATED)
Continue without alert
Outbox Staging (DP-01 Pub-Sub)
INSERT OutboxEvent
event_type = VITAL_RECEIVED
(always staged)
Alert was created?
yes
no
INSERT OutboxEvent
event_type = ALERT_CREATED_NOTIFICATION
INSERT OutboxEvent
event_type = ALERT_CREATED_ESCALATION
INSERT AuditLog
(action = ALERT_EVENT_ENQUEUED)
COMMIT (atomic transaction:
VitalReading + Alert + OutboxEvents + AuditLogs)
Return VitalIngestResponse
{reading_id, alert_created,
risk_score, severity}
Background: Outbox Publisher
(separate thread, polls continuously)
SELECT OutboxEvent
WHERE status IN (PENDING, FAILED)
ORDER BY created_at
Pending events to publish?
no more
yes
Read next OutboxEvent
Wrap in event envelope
{schema_version, event_type,
occurred_at, data}
event_type
VITAL_RECEIVED
ALERT_CREATED_NOTIFICATION
ALERT_CREATED_ESCALATION
publish to vitaltrack.vitals.received
publish to vitaltrack.notifications.events
publish to vitaltrack.escalation.events
Broker ack received?
yes
no
UPDATE status = PUBLISHED,
published_at = now()
UPDATE status = FAILED,
retry_count++,
last_error
Sleep until next poll interval
Worker running?
yes

VitalTrack - Entity-Relationship Diagram (Information Viewpoint)




C4 Component Diagram (L3) - Ingestion Service
Ingestion Service [Container: FastAPI]
Inbound APIs
«component» Root Endpoint GET / [FastAPI route]
«component» Registration Endpoint POST /auth/register [FastAPI route]
«component» Auth Token Endpoint POST /auth/token [FastAPI route]
«component» Vital Ingestion Endpoint POST /v1/vitals [FastAPI route]
get_password_hash
authenticate_user, create_access_token
require_roles (SIMULATOR, DOCTOR, ADMIN)
create(source)
normalize payload
creates default subject (at startup)
Cross-Cutting
«component» Auth Module authenticate_user, create_access_token, verify_password (bcrypt), require_roles(...) [auth.py]
Source Adapter (DP-03 + DP-04 Factory)
«component» VitalSourceAdapterFactory +create(source) -> VitalSourceAdapter
«component» VitalSourceAdapter (Protocol) +adapt(payload): dict [ingestion_main.py]
«creates if source=simulator»
«creates»
«component» SimulatorVitalSourceAdapter [forces source = 'simulator']
«component» GenericVitalSourceAdapter [default normalization]
DP-05 Observer Pipeline (in-process)
«component» build_default_vital_subject factory function wiring the default observer set [vital_observer.py]
notify(context)
returns wired subject
«component» VitalSubject attach(observer), notify(context) [vital_observer.py]
update(context)
update(context)
«component» RiskScoreObserver .update() -> calls Risk Score Service [vital_observer.py]
«component» AlertEvaluationObserver .update() -> calls Alert Rule Chain [vital_observer.py]
calculate_risk_score(reading)
evaluate_alert(reading, score)
context.risk_score = ...
context.severity, rule_code, message
«component» VitalObservationContext shared dataclass (reading, risk_score, severity, rule_code, message) [vital_observer.py]
Domain Logic
«component» Audit Service create_audit_log() [services.py]
«component» Risk Score Service calculate_risk_score(reading) [services.py]
«component» Alert Rule Chain (Chain of Responsibility) CriticalVitalsRule -> HighRiskScoreRule -> AbnormalTrendRule [services.py]
Transactional Outbox (ADR-008)
«component» Outbox Publisher Worker _publish_pending_outbox() background thread [ingestion_main.py]
«component» Outbox Writer INSERT OutboxEvent rows in same transaction as domain changes [ingestion_main.py]
«component» Event Envelope Builder build_event_envelope(event_type, data, schema_version) [messaging.py]
publish_event(payload, queue)
wrap event
INSERT User, Patient, CaregiverLinkRequest
ALERT_CREATED audit
SELECT User, verify password
COMMIT (atomic)
INSERT VitalReading
INSERT Alert if should_alert)
enqueue VITAL_RECEIVED + optional ALERT_CREATED_*
INSERT AuditLog
SELECT pending OutboxEvent
UPDATE status (PUBLISHED / FAILED)
INSERT OutboxEvent
Infrastructure
«component» DB Module engine, SessionLocal, get_db dependency [db.py]
«component» ORM Models User, Patient, VitalReading, Alert, OutboxEvent, AuditLog, CaregiverAssignment, ... [models.py]
«component» RabbitMQBridge publish_event(payload, queue_name) start/stop_consumer() DLX/DLQ setup [messaging.py]
SQLAlchemy / psycopg2
AMQP publish
PostgreSQL + TimescaleDB
«broker» RabbitMQ (vitals + notifications + escalation queues)



Task 4: Prototype Implementation and Analysis

4.1 Prototype Development (Implemented Scope)

The VitalTrack prototype is implemented as a coarse-grained microservices architecture using an event-driven communication model, aligned with the architectural decisions defined earlier. The system is deployed using Docker Compose and demonstrates realistic interactions across multiple backend services during controlled prototype evaluation.

Prototype Vitals Note
For prototype validation, VitalTrack uses a fixed core vital set: heart rate (bpm), SpO2 (%), systolic/diastolic blood pressure (mmHg), respiratory rate (breaths/min), temperature (C) , etc. These inputs are generated by the Device Simulator and manual UI entry to exercise ingestion, alerting, prediction, escalation, and chatbot advisory workflows under controlled conditions. The selected set is intentionally minimal for demonstrability and can be extended in production to include additional clinical signals and device-native telemetering.

The following core components are implemented:

- Ingestion Service: Handles vital data ingestion, validation, normalization, threshold-based alert generation, transactional outbox processing, and authentication token issuance.
- Notification Service: Manages alert lifecycle, notification delivery, WebSocket-based real-time updates, dashboards, and the chatbot endpoint.
- Prediction Service: Consumes vital events and applies Early Warning Score (EWS) and personalized baseline strategies to generate advisory risk predictions.
- Escalation Mediator Service: Coordinates multi-step escalation workflows for unacknowledged critical alerts.
- API Gateway (NGINX): Provides a single external entry point and routes requests to backend services.
- RabbitMQ Broker: Enables asynchronous communication with support for dead-letter queues (DLQ).
- PostgreSQL with TimescaleDB: Stores persistent relational and time-series data.
- React Frontend: Provides role-based dashboards and user interfaces.

The prototype demonstrates multiple end-to-end flows:

Critical Alert Pipeline:
Vital ingestion → threshold evaluation in Ingestion + supplementary prediction-based risk alerts from Prediction/Notification path → alert persistence → event publication → notification processing → WebSocket dashboard update → audit log
Clinical Acknowledgment Pipeline:
Doctor acknowledges alert → state update → WebSocket broadcast → audit entry
Critical Escalation Pipeline:
Unacknowledged alert → escalation steps (doctor → on-call → caregiver → admin) → notifications → audit trail
Chatbot Advisory Pipeline:
Chatbot request → triage (OpenAI/ Gemini + fallback) → advisory response → audit entry (no alert triggered)

4.2 Architecture Analysis (Comparison Against Another Pattern)

This is the evaluation of the chosen architecture against a realistic alternative (a modular monolith with in-process orchestration) with respect to the system's architecturally significant requirements and quality attribute scenarios.

Comparison:

Criterion	Microservices + Event-Driven (Chosen)	Modular Monolith	Winner for VitalTrack
Latency (QAS-01)	Higher per-event overhead (~50–100 ms broker hop + outbox delay)	Lower (in-process calls)	Monolith (but MS meets targets: p95 ≤5s)
Scalability (NFR-05)	Services scale independently (ingestion, prediction, notification)	Scales as a single unit	Microservices
Fault Isolation (QAS-02)	Broker outage → ingestion continues via outbox	Single failure domain	Microservices
Modifiability (QAS-05)	New consumer = zero changes to existing services	Changes across shared modules	Microservices
Operational Complexity	Multiple services + broker + DB + gateway	Single deployable	Monolith
Debugging	Distributed tracing required	Simple stack traces	Monolith
Evolvability (ASR-04, ASR-05)	Prediction & chatbot evolve independently	Coupled release cycle	Microservices

Reasoning

- **QAS-01 (Alert Latency):**
  The event-driven path introduces additional asynchronous overhead (broker transit, consumer scheduling, and outbox polling), typically in the tens of milliseconds range under this prototype deployment. In practice, measured end-to-end latency remains well below target (median 0.211s, p95 0.227s), so the overhead is acceptable for the prototype.
- **QAS-02 (Failure Recovery):**
  In the chosen architecture, ingestion continues during broker outages via the transactional

outbox, with events delivered after recovery. In a monolith, message delivery is tightly coupled to execution flow, increasing the risk of silent failures or blocking behavior.

- **QAS-05 (Adding Notification Channel):**
  In microservices, a new notification consumer can be added without modifying ingestion or existing services. In a monolith, notification logic is typically embedded in shared modules, requiring coordinated changes and redeployment.

Why Microservices + EDA is Preferred for VitalTrack :

ASR-04 (Modifiability) and ASR-05 (Prediction Evolvability) are central to VitalTrack. Prediction models, chatbot logic, and escalation workflows are expected to evolve independently. A modular monolith would force all subsystems into a shared deployment cycle, slowing iteration and increasing risk.

The chosen architecture isolates these concerns:

- Prediction logic evolves independently of ingestion and notification.
- New consumers (analytics, ML models) can be added without modifying existing code.
- Failure isolation ensures ingestion continues even when downstream components fail.

The trade-off is increased operational complexity, but this is justified because measured performance (§4.3) shows that system overhead does not impact user-facing latency.

The chosen architecture supports independent scaling and evolution of subsystems such as prediction and escalation, which is important for long-term system growth.

4.3 Quantified NFR Results

Test Environment

The system was tested in a single-host Docker Compose environment with all services running locally. Host system: standard development machine (CPU and memory sufficient for all containers), with no competing workloads during testing.

- Latency measurements: 100 sequential critical-vital ingestion requests, measured end-to-end using a Python client (`time.perf_counter()`) from HTTP request to WebSocket event receipt.
- Throughput measurements: 10 concurrent clients sending continuous vital data for 30 seconds using an asynchronous load script.

NFR	Metric	Measured Result	Target	Status
NFR-01 Performance	End-to-end alert latency (ingest → WebSocket)	Median 0.211s, p95 0.227s (100 samples)	≤3s / ≤5s (prototype)	Pass
NFR-01 Performance	API response time (POST /v1/vitals)	Median 0.109s, p95 0.162s	p95 ≤1s	Pass
NFR-05 Scalability	Ingestion throughput (10 clients, 30s)	87.66 req/s (2638/2638 success)	Prototype: measured/report observed max sustained throughput	Achieved
NFR-02 Reliability	Error rate under load	0.0% (0 errors)	Stable operation	Pass
NFR-02 Reliability	Poison message handling	DLQ triggered on malformed input	Failure isolation	Pass
NFR-07 Testability	Test suite pass rate	13/13 tests passed	≥1 integration per E2E + ≥1 unit per critical function	Pass
NFR-02 Reliability	Broker outage recovery (QAS-02)	RabbitMQ stopped for 60s during ingestion (~5 readings/s). Ingestion persisted successfully; outbox backlog drained in **5.36**	Undelivered events processed within 60s of recovery; zero alert-event loss	Pass

		seconds after broker recovery; undelivered events processed.		
NFR-07 Testability	Line coverage	~40% (1428 stmts, 916 miss)	≥80% (production) / meaningful prototype	Acknowledged gap

Analysis

The prototype demonstrates strong performance, with sub-second end-to-end latency and stable throughput under concurrent load. Reliability mechanisms such as the transactional outbox and DLQ ensure zero data loss and proper failure isolation.

While test coverage is relatively low (~40%), it is intentionally concentrated on critical paths such as alert generation, event delivery, access control, and end-to-end flows, aligning with the prototype's validation goals.

A broker-outage recovery drill was executed by stopping RabbitMQ for 60 seconds while continuing ingestion. During outage, ingestion remained available and events accumulated in the transactional outbox. After broker restart, backlog publishing resumed automatically and drained within the measured recovery window 5.36s , with no observed alert-event loss. This directly validates the NFR-02 recovery scenario.

4.4 Trade-Off Discussion

Performance vs Decoupling

The event-driven architecture introduces overhead from broker communication and outbox polling. In practice, this adds tens to hundreds of milliseconds per event. However, measured latency remains well within targets, making this overhead negligible for users.

The benefit is significant: ingestion is decoupled from downstream services, allowing the system to continue accepting data even during broker outages.

Reliability vs Complexity

The combination of transactional outbox and dead-letter queue introduces additional components (outbox table, background publisher, DLQ configuration).

This complexity is justified because it eliminates the dual-write failure scenario, where alerts could be stored in the database but never published to the broker. Testing confirmed that without these mechanisms, broker outages could lead to silent data loss.

Modifiability vs Operational Cost

Separating Prediction and Escalation into independent services improves locality and allows new features to be added without modifying existing subsystems.

The trade-off is increased operational overhead (more services, logs, and health checks), which is acceptable given the system's need for long-term extensibility.

Scalability vs Simplicity

The architecture allows independent scaling of ingestion, prediction, and notification workloads. For example, prediction workloads can increase without affecting ingestion throughput.

A monolithic design would be simpler but would require scaling the entire system even when only one subsystem is under load.

Observability Requirements

The distributed design requires stronger monitoring, including:

- Queue depth and DLQ activity
- Outbox backlog
- Service health endpoints

This increases operational responsibility but is necessary to maintain visibility and reliability in a distributed system.

4.5 Security and Access-Control Validation

The system enforces authentication and authorization across all protected endpoints. JWT-based authentication is required for API access, and role-based authorization is enforced server-side for ADMIN, DOCTOR, PATIENT, CAREGIVER, and SIMULATOR roles.

Sensitive operations—including patient data access, administrative actions, alert handling, and audit retrieval—are protected through role checks and access-scope validation.

Caregiver access follows an admin-mediated approval workflow. A caregiver may request access to a patient, but access is only granted after administrator approval. Pending or rejected requests do not provide any access.

This ensures least-privilege access, prevents unauthorized data exposure, and supports auditability. The design satisfies NFR-03 (Security), NFR-04 (Privacy), FR-09, FR-10, and ADR-009.

Note: In the prototype, DOCTOR self-signup is allowed for demonstration convenience; in production, doctor account creation should be restricted to administrators or verified onboarding workflows. For prototype evaluation convenience, the `/auth/register` endpoint accepts DOCTOR role self-registration (see ADR-009 Consequences); production deployments restrict DOCTOR creation to administrator workflows.

4.6 Conclusion

The VitalTrack prototype successfully demonstrates an event-driven microservices architecture in practice. The system meets its functional requirements and achieves strong performance, reliability, and scalability results.

Measured metrics show sub-second latency, stable throughput, and effective failure isolation using transactional outbox and DLQ mechanisms. Compared to a modular monolith, the architecture offers better flexibility, scalability, and fault isolation, although at the cost of increased operational complexity.

Given the need to support evolving prediction models and escalation workflows, the chosen architecture provides a solid and extensible foundation for future development.

The following architecture artifacts are included in the submission folder / github :

01_c4_context.puml, 02_c4_container.puml, 03_deployment_diagram.puml, 04_usecase_diagram.puml, 05_usecase_monitoring_alerts.puml, 06_class_diagram.puml, 07_sequence_vital_ingestion.puml, 08_sequence_escalation.puml, 09_sequence_chatbot.puml, 10_class_diagram_publish_subscribe.puml, 11_class_diagram_strategy.puml, 12_class_diagram_adapter.puml, 13_class_diagram_factory.puml, 14_class_diagram_observer.puml, 15_access_matrix.md, 16_er_diagram.puml, 17_state_escalation_case.puml, 18_activity_vital_ingestion.puml, 19_activity_escalation.puml, 20_c4_component_notification.puml, 21_c4_component_ingestion.puml, 22_c4_code_notification_chatbot.puml, 23_c4_code_ingestion_pipeline.puml, and 24_views_viewpoints_mapping.md.

5. CONTRIBUTIONS:

Kashik P S: Task 1 NFRs & Metrics , Quality Attribute Scenarios , ASR,Subsystem Overview ; Task 2 ADRs , Architectural Views & Viewpoints (some part) , Stakeholder Views Concern mapping, ; Task 3 Tactics & Patterns ; Task 4 Prototype Implementation & Integration of *COMPLETE* 5 E2E functionalities & services & full codebase with README & Enhancement of Chatbot & Added Tests , Architectural Analysis & Comparison , NFR Metrics Measured Results , Tradeoffs , Activity , State, ER diagrams & All diagrams consistency verification . Full project verification & checking

Piyush : Task 1 Subsystem Overview , Task 2 Stakeholders & Concerns & Architectural Views & Viewpoints & 1 Diagram related to it .

Ansh : Task 1 FRs , Subsystem Overview & Verification

Nikhil :Task 1 Subsystem Overview, Task 2 & 3 Diagrams , Task 3 Tactics and Patterns, Verification

Harsh: Task 2 Stakeholders , Chatbot (Feature) , Verification

THANKS